

Wright Mediation Framework: Effect size metric, Variance Decomposition, and Power/Sensitivity Analysis

A Unified Approach for Complex Mediation Designs

mediationgourp

2026-06-24

Table of contents

1. Introduction to the Effect Size Metric	2
1.1 The Need for a Standardized Effect Size Metric in Mediation	2
1.2 Sewall Wright's Path Tracing Rules	2
1.3 The Coefficient of Determination (R^2) as an Effect Size	2
2. Theoretical Foundations of R^2_{indirect}	3
2.1 Mathematical Formulation	3
2.2 Algorithmic Implementation	3
3. Algebraic Validation of the Metric (<code>wright_proof</code>)	3
3.1 Mathematical Derivation of the 5-Block Decomposition	3
3.2 Statistical Justification of the Joint Covariance Term	4
3.3 Interpretation Under Suppression Conditions	4
3.4 Scaling the Decomposition for Complex Models	4
4. The R^2_{indirect} Function (<code>r2_indirect</code>)	5
4.1 Applying the function to different mediation models	8
4.1.1 Single Mediator Models	8
4.1.2 Models with Two Parallel Mediators	54
4.1.3 Models with Two Serial Mediators	103
5. A-Priori Power Analysis and Sensitivity Analysis based on R^2_{indirect}	163
5.1 Theoretical Framework: Joint Significance Power	163
5.2 Mathematical Boundary Profiles: Balanced vs. Bottleneck Scenarios	163
5.3 Error Variance Adjustments and Multi-Mediator Penalties	164
5.4 The Empirical Guardrail (<code>max_beta</code>)	164
6. Power Analysis for Single, Parallel, and Serial Mediation (<code>wright_power</code>)	165
6.1 Algorithmic Mechanics	165
6.2 Structural Adaptation (Single, Parallel, and Serial Models)	165
7. Sensitivity Analysis for the MDES (<code>wright_sensitivity</code>)	171

7.1 Algorithmic Mechanics	171
7.2 Structural Interpretation of Outputs	172

1. Introduction to the Effect Size Metric

1.1 The Need for a Standardized Effect Size Metric in Mediation

In structural equation modeling (SEM) and mediation analysis, quantifying the magnitude of an indirect effect relies primarily on the product of standardized path coefficients ($\beta_a \cdot \beta_b$). Although this approach standardizes the effect by expressing it in standard deviation units, the resulting product functions as a metric of relative linear change rather than a bounded proportion of shared variance. Consequently, it does not directly reflect the operational contribution of the mediation pathway to the total variance of the outcome variable. This limitation makes it difficult to evaluate the practical significance of the mechanism within an absolute, variance-accounted-for framework that is directly comparable to traditional effect sizes .

1.2 Sewall Wright's Path Tracing Rules

To derive a formalized effect size, this framework leverages Sewall Wright's seminal path tracing rules. These foundational principles state that the total variance of an outcome variable can be exactly decomposed into the sum of the structural pathways connecting the variable to itself, either through direct, indirect, or common causal tracks.

In a standard single-mediator structure where an independent variable (X) affects the dependent variable (Y) directly via path c' and indirectly through a mediator (M) via paths a and b , Wright's rules partition the total observed variance of the outcome (σ_Y^2) into the following additive components:

$$\sigma_Y^2 = c'^2\sigma_X^2 + a^2b^2\sigma_X^2 + 2abc'\sigma_X^2 + b^2\sigma_{e_M}^2 + \sigma_{e_Y}^2$$

Where: * $c'^2\sigma_X^2$ represents the variance uniquely explained by the direct effect of X . * $a^2b^2\sigma_X^2$ represents the variance uniquely explained by the indirect pathway ($X \rightarrow M \rightarrow Y$). * $2abc'\sigma_X^2$ captures the unanalyzed covariance between the direct and indirect paths arising from their shared upstream cause (X). * $b^2\sigma_{e_M}^2$ represents the variance transmitted to Y from the unexplained residual of the mediator (e_M). * $\sigma_{e_Y}^2$ is the residual error variance intrinsic to Y .

By systematically accounting for every linear component, this exact decomposition provides the mathematical foundation necessary to isolate the unique variance explained (R^2) by the indirect pathway without confounding or double-counting effects across multi-stage structures.

1.3 The Coefficient of Determination (R^2) as an Effect Size

Building upon Wright's decomposition, this framework proposes evaluating the indirect effect as a strict proportion of the outcome's variance. By expressing the structural mechanism as a Coefficient of Determination (R_{indirect}^2), the indirect effect is mapped onto a bounded scalar ranging from 0 to 1. This provides a standardized metric representing the specific amount of variance the mediation pathway explains within the final outcome.

2. Theoretical Foundations of R^2_{indirect}

2.1 Mathematical Formulation

The `r2_indirect` function calculates the proportion of variance explained by specific indirect paths. For a single mediation model defined by unstandardized path coefficients a and b , the variance of the independent variable σ_X^2 , and the variance of the dependent variable σ_Y^2 , the metric is algebraically formulated as:

$$R^2_{\text{indirect}} = \frac{(a \cdot b)^2 \cdot \sigma_X^2}{\sigma_Y^2}$$

2.2 Algorithmic Implementation

The custom R function extracts the unstandardized point estimates and the implied covariance matrix directly from a fitted structural model. It computes the squared product of the targeted indirect paths and scales it by the ratio of the predictor's variance to the outcome's variance. To provide a measure of precision and statistical inference, the function utilizes a non-parametric bootstrapping procedure (`n_boot`) to compute 95% Confidence Intervals for the estimated variance proportion.

3. Algebraic Validation of the Metric (`wright_proof`)

To validate the proposed metric algebraically, the framework includes the `wright_proof` suite. This validation ensures that the variance components extracted from the structural paths map accurately onto the theoretically implied variance of the outcome variable, guaranteeing mathematical consistency.

3.1 Mathematical Derivation of the 5-Block Decomposition

Consider a simple mediation model defined by the following structural equations, where all variables are expressed as deviations from their means:

$$M = aX + e_M$$

$$Y = c'X + bM + e_Y$$

Under standard ordinary least squares (OLS) and SEM assumptions, the independent variable X , the mediator residual e_M , and the outcome residual e_Y are mutually independent:

$$(\text{Cov}(X, e_M) = \text{Cov}(X, e_Y) = \text{Cov}(e_M, e_Y) = 0)$$

Substituting the equation for M into the equation for Y isolates the independent sources of variance:

$$Y = c'X + b(aX + e_M) + e_Y$$

$$Y = (c' + ab)X + be_M + e_Y$$

Applying the variance operator (Var) to both sides of the equation yields:

$$\text{Var} (Y) = \text{Var} ((c' + ab)X) + \text{Var} (be_M) + \text{Var} (e_Y)$$

Extracting the constant coefficients from the variance operator results in:

$$\sigma_Y^2 = (c' + ab)^2 \sigma_X^2 + b^2 \sigma_{e_M}^2 + \sigma_{e_Y}^2$$

Expanding the binomial term $(c' + ab)^2 = (c')^2 + (ab)^2 + 2c'ab$ provides the exact 5-block decomposition implemented in the verification script:

$$\sigma_Y^2 = (c')^2 \sigma_X^2 + (ab)^2 \sigma_X^2 + 2c'ab \sigma_X^2 + b^2 \sigma_{e_M}^2 + \sigma_{e_Y}^2$$

Where the five distinct components are defined as: 1. **Pure Direct Effect:** $(c')^2 \cdot \sigma_X^2$ 2. **Pure Indirect Effect:** $(a \cdot b)^2 \cdot \sigma_X^2$ 3. **Joint Covariance:** $2 \cdot c' \cdot a \cdot b \cdot \sigma_X^2$ 4. **Independent Mediator Error:** $b^2 \cdot \sigma_{e_M}^2$ 5. **Residual Outcome Error:** $\sigma_{e_Y}^2$

3.2 Statistical Justification of the Joint Covariance Term

The inclusion of the third component, $2c'ab\sigma_X^2$, is required by the probability theorem for the variance of the sum of two random variables. For any two random variables A and B :

$$\text{Var} (A + B) = \text{Var} (A) + \text{Var} (B) + 2 \cdot \text{Cov} (A, B)$$

In this structural context, let $A = c'X$ and $B = abX$. The covariance between these two linear combinations is derived as:

$$\text{Cov} (c'X, abX) = c' \cdot ab \cdot \text{Cov} (X, X) = c'ab\sigma_X^2$$

Therefore, the joint covariance term is an essential algebraic component of the total variance of Y . Omitting this term would violate principles of linear algebra and result in an incorrect reconstruction of the observed variance.

3.3 Interpretation Under Suppression Conditions

The decomposition accounts for statistical suppression effects through the sign of the Joint Covariance block. When the direct effect (c') and the indirect effect ($a \cdot b$) have opposing signs, the product $c'ab$ is negative, causing the entire joint covariance term to become negative ($2c'ab\sigma_X^2 < 0$).

In this scenario, the negative term subtracts from the total implied variance, modeling how the opposing pathways offset one another. To maintain the algebraic equality with the total observed variance σ_Y^2 , the residual outcome error variance ($\sigma_{e_Y}^2$) must expand. This mathematically reflects the operational reality of suppression designs, where opposing paths reduce the overall explained variance of the model.

3.4 Scaling the Decomposition for Complex Models

The algebraic validation scales to accommodate more complex mediation architectures. For instance:

- **Two Parallel Mediators:** The variance of the outcome variable is partitioned into 10 distinct blocks. This structure accounts for the unanalyzed residual covariance between parallel mediators ($2 \cdot b_1 \cdot b_2 \cdot \text{Cov}(e_{M1}, e_{M2})$) and the multi-way joint structural covariances.
- **Two Serial Mediators:** The structure expands to 15 distinct components, tracking the sequential variance flow through the internal path connecting the mediators (d_{21}).

In all configurations, the `wright_proof` function verifies that the final residual error block remains non-negative and that the total decomposed variance precisely matches the observed variance of the outcome. This procedure may be extended to even more complex models.

4. The R^2_{indirect} Function (`r2_indirect`)

#FUNCTION FOR ESTIMATING SPECIFIC VARIANCE EXPLAINED BY INDIRECT EFFECTS

```
r2_indirect <- function(model_fit, x_var, y_var, ..., n_boot = 500) {

  # -----
  # DEPENDENCY MANAGEMENT
  # Check if lavaan is loaded; if not, try to load it dynamically.
  # -----
  if (!"package:lavaan" %in% search()) {
    if (!require("lavaan", character.only = TRUE, quietly = TRUE)) {
      stop("Error: The 'lavaan' package is required but not installed. Please
run install.packages('lavaan') first.")
    } else {
      message("Note: 'lavaan' package has been automatically loaded by
r2_indirect().")
    }
  }

  # Capture the paths passed directly via ...
  paths_list <- list(...)
  num_paths <- length(paths_list)

  if (num_paths == 0) {
    stop("Error: Please specify at least one mediation path (e.g., path1 =
c('a', 'b')).")
  }

  # Internal function for point estimates
  get_indices <- function(fit) {
    param <- parameterEstimates(fit)
    cov_matrix <- lavInspect(fit, "cov.ov")
    vX <- cov_matrix[x_var, x_var]
    vY <- cov_matrix[y_var, y_var]
```

```

product_effects <- numeric(num_paths)

for(j in 1:num_paths) {
  path_labels <- paths_list[[j]]
  # Extract and multiply the coefficients for the given path
  coef_values <- sapply(path_labels, function(l) param$est[param$label == l])
  product_effects[j] <- prod(coef_values)
}

# Calculate specific R2 indices
r2_specific <- (product_effects)^2 * vX / vY

# Calculate R2 for the TOTAL MEDIATED EFFECT
r2_total <- (sum(product_effects))^2 * vX / vY

return(c(r2_specific, r2_total))
}

# Point estimates on original data
point_estimates <- get_indices(model_fit)

# Bootstrapping procedure
cat("Running Bootstrap on", num_paths, "specific path(s) + Total Mediated
Effect...\n")
orig_data <- as.data.frame(lavInspect(model_fit, "data"))
boot_matrix <- matrix(NA, nrow = n_boot, ncol = num_paths + 1)

pb <- txtProgressBar(min = 0, max = n_boot, style = 3)
for(i in 1:n_boot) {
  boot_idx <- sample(1:nrow(orig_data), replace = TRUE)
  boot_data <- orig_data[boot_idx, ]
  fit_boot <- try(suppressWarnings(sem(model = model_fit@ParTable, data =
boot_data)), silent = TRUE)

  if(!inherits(fit_boot, "try-error") && lavInspect(fit_boot, "converged")) {
    boot_matrix[i, ] <- get_indices(fit_boot)
  }
  setTxtProgressBar(pb, i)
}
close(pb)

# Clean NA rows (non-converged bootstraps)

```

```

boot_matrix <- na.omit(boot_matrix)

# 95% Confidence Intervals
ci_low <- apply(boot_matrix, 2, quantile, probs = 0.025)
ci_high <- apply(boot_matrix, 2, quantile, probs = 0.975)

# Format output names nicely based on user input
path_names <- names(paths_list)
if (is.null(path_names)) {
  path_names <- paste0("Path_", 1:num_paths, " [", sapply(paths_list, paste,
collapse="*"), "]")
} else {
  empty_names <- path_names == ""
  path_names[empty_names] <- paste0("Path_", which(empty_names))
  path_names <- paste0(path_names, " [", sapply(paths_list, paste,
collapse="*"), "]")
}

effect_names <- c(path_names, "Total Mediated Effect")

# Build final dataframe
final_report <- data.frame(
  Effect = effect_names,
  r2_indirect = round(point_estimates, 5),
  CI_Low = round(ci_low, 5),
  CI_High = round(ci_high, 5),
  Perc = paste0(round(point_estimates * 100, 2), "%"),
  CI_Low_P = paste0(round(ci_low * 100, 2), "%"),
  CI_High_P = paste0(round(ci_high * 100, 2), "%")
)

return(final_report)
}

```

###EXAMPLE OF USAGE: results <- r2_indirect(model_fit = lavaan.mod.fit, x_var = "X", y_var = "Y", ###User defined paths (Specify any type of indirect association as defined in the fitted lavaan model)### Single = c("path_a", "path_b"), Parallel1 = c("path_a1", "path_b1"), Parallel2 = c("path_a2", "path_b2"), Serial = c("path_a", "path_b", "path_d"), n_boot = 500)

print(results, row.names = FALSE)

4.1 Applying the function to different mediation models

4.1.1 Single Mediator Models

```
#
=====
# DATA GENERATION: SINGLE MEDIATOR
#
=====
simulate_data_single <- function(N, a, b, cp, seed = 123) {
  set.seed(seed)

  X <- rnorm(N, mean = 0, sd = 1)

  # Mediator with residual variance = 1
  eM <- rnorm(N, mean = 0, sd = 1)
  M <- a * X + eM

  # Outcome with residual variance = 1
  eY <- rnorm(N, mean = 0, sd = 1)
  Y <- cp * X + b * M + eY

  return(data.frame(X = X, M = M, Y = Y))
}

# -----
# Creation of the Datasets
# -----

# Dataset 1: N = 400 (Moderate positive effects)
df1 <- simulate_data_single(N = 400, a = 0.5, b = 0.5, cp = 0.4)

# Dataset 2: N = 500 (Strong positive effects)
df2 <- simulate_data_single(N = 500, a = 0.7, b = 0.6, cp = 0.3)

# Dataset 3: N = 600 (Mixed positive effects)
df3 <- simulate_data_single(N = 600, a = 0.4, b = 0.8, cp = 0.5)

# Dataset 4: N = 600 (TYPE 1 SUPPRESSION EFFECT: DIRECT NEGATIVE VS. INDIRECT
POSITIVE)
# X depresses Y (negative cp), but the mediator increases Y
df4 <- simulate_data_single(N = 600, a = 0.6, b = 0.6, cp = -0.7)

# Dataset 4: N = 600 (TYPE 2 SUPPRESSION EFFECT: DIRECT POSITIVE VS. INDIRECT
```



```

NEGATIVE)
# M depresses Y (negative ind effect), but the mediator increases Y
df5 <- simulate_data_single(N = 600, a = 0.6, b = -0.6, cp = 0.7)

#
=====
# LAVAAN MODEL DEFINITION
#
=====
# Load the required package
library(lavaan)

```

This is lavaan 0.6-21
lavaan is FREE software! Please report any bugs.

```

mod_simple <- "

# Paths
M ~ a * X
Y ~ cp * X + b * M

#Indirect effect
ind := a*b

"

fit1 <- sem(mod_simple, estimator = "MLR", data = df1)
fit2 <- sem(mod_simple, estimator = "MLR", data = df2)
fit3 <- sem(mod_simple, estimator = "MLR", data = df3)
fit4 <- sem(mod_simple, estimator = "MLR", data = df4)
fit5 <- sem(mod_simple, estimator = "MLR", data = df5)

summary(fit1, standardized = TRUE)

```

lavaan 0.6-21 ended normally after 1 iteration

Estimator	ML
Optimization method	NLMINB
Number of model parameters	5

Number of observations				400			
Model Test User Model:							
				Standard		Scaled	
Test Statistic				0.000		0.000	
Degrees of freedom				0		0	
Parameter Estimates:							
Standard errors				Sandwich			
Information bread				Observed			
Observed information based on				Hessian			
Regressions:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
M ~							
X	(a)	0.528	0.058	9.139	0.000	0.528	0.458
Y ~							
X	(cp)	0.341	0.056	6.126	0.000	0.341	0.259
M	(b)	0.495	0.047	10.480	0.000	0.495	0.433
Variances:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M		0.985	0.069	14.225	0.000	0.985	0.790
.Y		1.047	0.074	14.202	0.000	1.047	0.643
Defined Parameters:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
ind		0.261	0.040	6.551	0.000	0.261	0.198

```
summary(fit2, standardized = TRUE)
```

lavaan 0.6-21 ended normally after 1 iteration

Estimator	ML
Optimization method	NLMINB
Number of model parameters	5
Number of observations	500
Model Test User Model:	
	Standard Scaled

Test Statistic				0.000		0.000	
Degrees of freedom				0		0	
Parameter Estimates:							
Standard errors						Sandwich	
Information bread						Observed	
Observed information based on						Hessian	
Regressions:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
M ~							
X	(a)	0.646	0.047	13.794	0.000	0.646	0.528
Y ~							
X	(cp)	0.326	0.052	6.232	0.000	0.326	0.227
M	(b)	0.653	0.046	14.264	0.000	0.653	0.557
Variances:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M		1.017	0.063	16.171	0.000	1.017	0.721
.Y		0.978	0.063	15.566	0.000	0.978	0.504
Defined Parameters:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
ind		0.422	0.042	10.062	0.000	0.422	0.294

```
summary(fit3, standardized = TRUE)
```

lavaan 0.6-21 ended normally after 1 iteration

Estimator	ML	
Optimization method	NLMINB	
Number of model parameters	5	
Number of observations	600	
Model Test User Model:		
	Standard	Scaled
Test Statistic	0.000	0.000
Degrees of freedom	0	0
Parameter Estimates:		

Standard errors
Information bread
Observed information based on

Sandwich
Observed
Hessian

Regressions:

		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
M ~							
X	(a)	0.437	0.044	9.968	0.000	0.437	0.381
Y ~							
X	(cp)	0.419	0.045	9.326	0.000	0.419	0.266
M	(b)	0.853	0.040	21.224	0.000	0.853	0.621

Variances:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M	1.049	0.060	17.472	0.000	1.049	0.855
.Y	0.971	0.056	17.342	0.000	0.971	0.419

Defined Parameters:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
ind	0.373	0.041	9.059	0.000	0.373	0.236

```
summary(fit4, standardized = TRUE)
```

lavaan 0.6-21 ended normally after 1 iteration

Estimator ML
Optimization method NLMINB
Number of model parameters 5

Number of observations 600

Model Test User Model:

	Standard	Scaled
Test Statistic	0.000	0.000
Degrees of freedom	0	0

Parameter Estimates:

Standard errors
Information bread
Observed information based on

Sandwich
Observed
Hessian

Regressions:

		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
M ~							
X	(a)	0.637	0.044	14.534	0.000	0.637	0.515
Y ~							
X	(cp)	-0.792	0.049	-16.160	0.000	-0.792	-0.614
M	(b)	0.653	0.040	16.250	0.000	0.653	0.627

Variances:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M	1.049	0.060	17.472	0.000	1.049	0.735
.Y	0.971	0.056	17.342	0.000	0.971	0.626

Defined Parameters:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
ind	0.416	0.038	10.890	0.000	0.416	0.323

```
summary(fit5, standardized = TRUE)
```

lavaan 0.6-21 ended normally after 1 iteration

Estimator	ML
Optimization method	NLMINB
Number of model parameters	5
Number of observations	600

Model Test User Model:

	Standard	Scaled
Test Statistic	0.000	0.000
Degrees of freedom	0	0

Parameter Estimates:

Standard errors	Sandwich
Information bread	Observed
Observed information based on	Hessian

Regressions:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
M ~						

X	(a)	0.637	0.044	14.534	0.000	0.637	0.515
Y ~							
X	(cp)	0.608	0.049	12.417	0.000	0.608	0.506
M	(b)	-0.547	0.040	-13.598	0.000	-0.547	-0.563

Variances:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M	1.049	0.060	17.472	0.000	1.049	0.735
.Y	0.971	0.056	17.342	0.000	0.971	0.720

Defined Parameters:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
ind	-0.348	0.035	-9.878	0.000	-0.348	-0.290

```
#
=====
# RUNNING r2_indirect ON RAW COEFFICIENTS (UNSTANDARDIZED b) SINGLE MEDIATOR
#
=====

cat("\n=== LAUNCHING r2_indirect ON RAW UNSTANDARDIZED DATA ===\n")
```

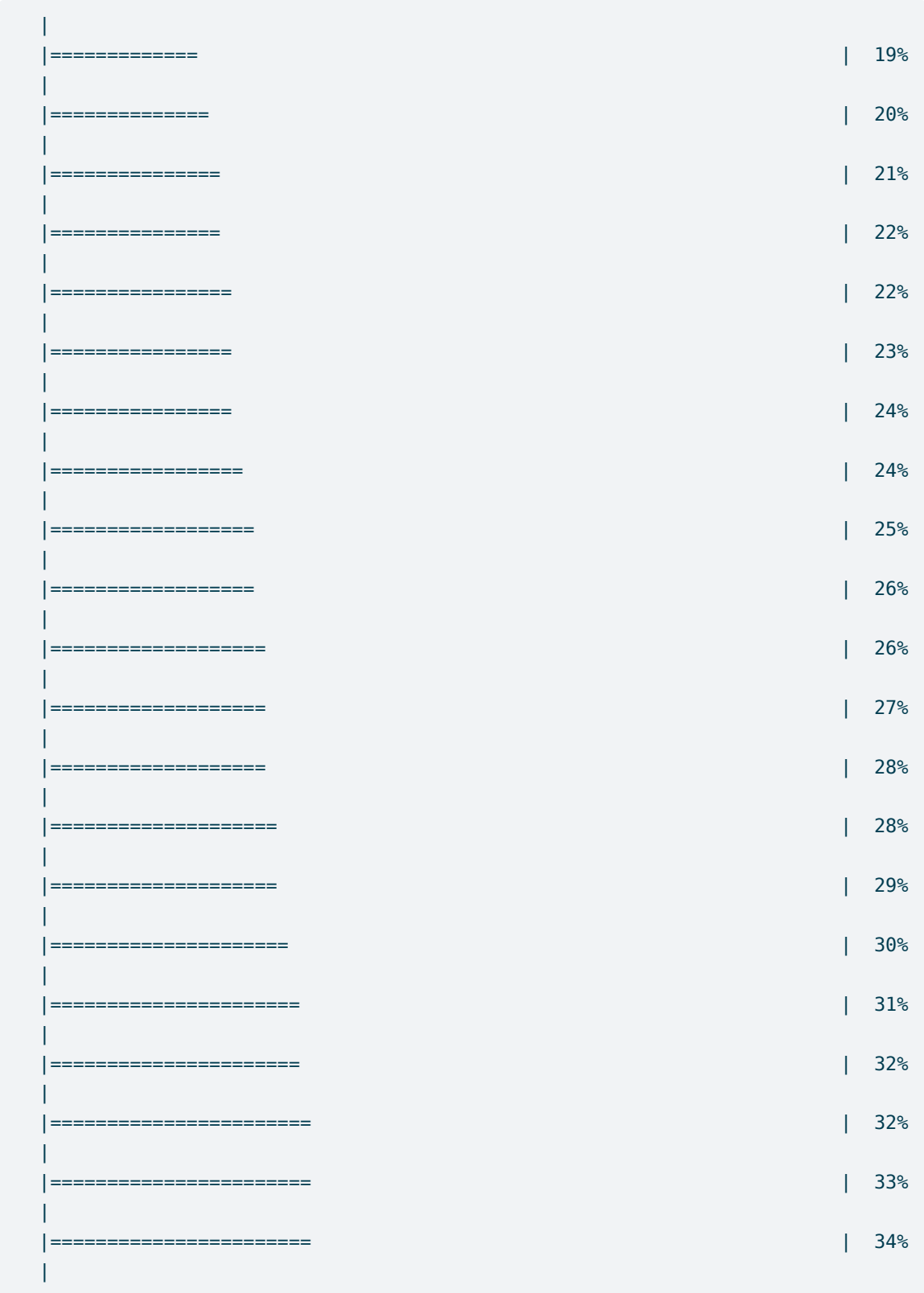
```
=== LAUNCHING r2_indirect ON RAW UNSTANDARDIZED DATA ===
```

```
res_df1 <- r2_indirect(fit1, x_var = "X", y_var = "Y", Med_Path = c("a", "b"),
n_boot = 200)
```

Running Bootstrap on 1 specific path(s) + Total Mediated Effect...

		0%
=		1%
=		2%
==		2%
==		3%

==		4%
===		4%
====		5%
=====		6%
=====		6%
=====		7%
=====		8%
=====		8%
=====		9%
=====		10%
=====		11%
=====		12%
=====		12%
=====		13%
=====		14%
=====		14%
=====		15%
=====		16%
=====		16%
=====		17%
=====		18%
=====		18%



=====	34%
=====	35%
=====	36%
=====	36%
=====	37%
=====	38%
=====	38%
=====	39%
=====	40%
=====	41%
=====	42%
=====	42%
=====	43%
=====	44%
=====	44%
=====	45%
=====	46%
=====	46%
=====	47%
=====	48%
=====	48%
=====	49%

=====		50%
=====		51%
=====		52%
=====		52%
=====		53%
=====		54%
=====		54%
=====		55%
=====		56%
=====		56%
=====		57%
=====		58%
=====		58%
=====		59%
=====		60%
=====		61%
=====		62%
=====		62%
=====		63%
=====		64%
=====		64%

=====	65%
=====	66%
=====	66%
=====	67%
=====	68%
=====	68%
=====	69%
=====	70%
=====	71%
=====	72%
=====	72%
=====	73%
=====	74%
=====	74%
=====	75%
=====	76%
=====	76%
=====	77%
=====	78%
=====	78%
=====	79%
=====	80%

=====		81%
=====		82%
=====		82%
=====		83%
=====		84%
=====		84%
=====		85%
=====		86%
=====		86%
=====		87%
=====		88%
=====		88%
=====		89%
=====		90%
=====		91%
=====		92%
=====		92%
=====		93%
=====		94%
=====		94%
=====		95%

=====	96%
=====	96%
=====	97%
=====	98%
=====	98%
=====	99%
=====	100%

```
res_df2 <- r2_indirect(fit2, x_var = "X", y_var = "Y", Med_Path = c("a", "b"),
n_boot = 200)
```

Running Bootstrap on 1 specific path(s) + Total Mediated Effect...

		0%
=		1%
=		2%
==		2%
==		3%
==		4%
===		4%
====		5%
====		6%
=====		6%
=====		7%

=====	8%
=====	8%
=====	9%
=====	10%
=====	11%
=====	12%
=====	12%
=====	13%
=====	14%
=====	14%
=====	15%
=====	16%
=====	16%
=====	17%
=====	18%
=====	18%
=====	19%
=====	20%
=====	21%
=====	22%
=====	22%
=====	23%

=====		24%
=====		24%
=====		25%
=====		26%
=====		26%
=====		27%
=====		28%
=====		28%
=====		29%
=====		30%
=====		31%
=====		32%
=====		32%
=====		33%
=====		34%
=====		34%
=====		35%
=====		36%
=====		36%
=====		37%
=====		38%

=====	38%
=====	39%
=====	40%
=====	41%
=====	42%
=====	42%
=====	43%
=====	44%
=====	44%
=====	45%
=====	46%
=====	46%
=====	47%
=====	48%
=====	48%
=====	49%
=====	50%
=====	51%
=====	52%
=====	52%
=====	53%
=====	54%

=====		54%
=====		55%
=====		56%
=====		56%
=====		57%
=====		58%
=====		58%
=====		59%
=====		60%
=====		61%
=====		62%
=====		62%
=====		63%
=====		64%
=====		64%
=====		65%
=====		66%
=====		66%
=====		67%
=====		68%
=====		68%

=====	69%
=====	70%
=====	71%
=====	72%
=====	72%
=====	73%
=====	74%
=====	74%
=====	75%
=====	76%
=====	76%
=====	77%
=====	78%
=====	78%
=====	79%
=====	80%
=====	81%
=====	82%
=====	82%
=====	83%
=====	84%
=====	84%

	=====	85%
	=====	86%
	=====	86%
	=====	87%
	=====	88%
	=====	88%
	=====	89%
	=====	90%
	=====	91%
	=====	92%
	=====	92%
	=====	93%
	=====	94%
	=====	94%
	=====	95%
	=====	96%
	=====	96%
	=====	97%
	=====	98%
	=====	98%
	=====	99%

===== 100%

```
res_df3 <- r2_indirect(fit3, x_var = "X", y_var = "Y", Med_Path = c("a", "b"),  
n_boot = 200)
```

Running Bootstrap on 1 specific path(s) + Total Mediated Effect...

		0%
=		1%
=		2%
==		2%
==		3%
==		4%
===		4%
====		5%
====		6%
=====		6%
=====		7%
=====		8%
=====		8%
=====		9%
=====		10%
=====		11%
=====		12%

=====		12%
=====		13%
=====		14%
=====		14%
=====		15%
=====		16%
=====		16%
=====		17%
=====		18%
=====		18%
=====		19%
=====		20%
=====		21%
=====		22%
=====		22%
=====		23%
=====		24%
=====		24%
=====		25%
=====		26%
=====		26%

=====	27%
=====	28%
=====	28%
=====	29%
=====	30%
=====	31%
=====	32%
=====	32%
=====	33%
=====	34%
=====	34%
=====	35%
=====	36%
=====	36%
=====	37%
=====	38%
=====	38%
=====	39%
=====	40%
=====	41%
=====	42%
=====	42%

=====		43%
=====		44%
=====		44%
=====		45%
=====		46%
=====		46%
=====		47%
=====		48%
=====		48%
=====		49%
=====		50%
=====		51%
=====		52%
=====		52%
=====		53%
=====		54%
=====		54%
=====		55%
=====		56%
=====		56%
=====		57%

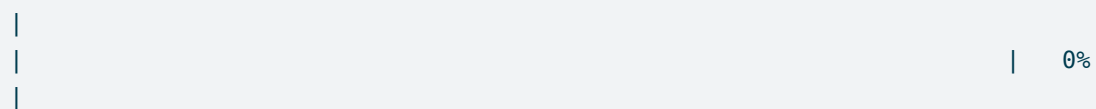
=====	58%
=====	58%
=====	59%
=====	60%
=====	61%
=====	62%
=====	62%
=====	63%
=====	64%
=====	64%
=====	65%
=====	66%
=====	66%
=====	67%
=====	68%
=====	68%
=====	69%
=====	70%
=====	71%
=====	72%
=====	72%
=====	73%

=====		74%
=====		74%
=====		75%
=====		76%
=====		76%
=====		77%
=====		78%
=====		78%
=====		79%
=====		80%
=====		81%
=====		82%
=====		82%
=====		83%
=====		84%
=====		84%
=====		85%
=====		86%
=====		86%
=====		87%
=====		88%



```
res_df4 <- r2_indirect(fit4, x_var = "X", y_var = "Y", Med_Path = c("a", "b"),
n_boot = 200)
```

Running Bootstrap on 1 specific path(s) + Total Mediated Effect...



=		1%
=		2%
==		2%
==		3%
==		4%
===		4%
====		5%
====		6%
=====		6%
=====		7%
=====		8%
=====		8%
=====		9%
=====		10%
=====		11%
=====		12%
=====		12%
=====		13%
=====		14%
=====		14%
=====		15%
=====		16%

=====		16%
=====		17%
=====		18%
=====		18%
=====		19%
=====		20%
=====		21%
=====		22%
=====		22%
=====		23%
=====		24%
=====		24%
=====		25%
=====		26%
=====		26%
=====		27%
=====		28%
=====		28%
=====		29%
=====		30%
=====		31%

=====	32%
=====	32%
=====	33%
=====	34%
=====	34%
=====	35%
=====	36%
=====	36%
=====	37%
=====	38%
=====	38%
=====	39%
=====	40%
=====	41%
=====	42%
=====	42%
=====	43%
=====	44%
=====	44%
=====	45%
=====	46%
=====	46%

=====		47%
=====		48%
=====		48%
=====		49%
=====		50%
=====		51%
=====		52%
=====		52%
=====		53%
=====		54%
=====		54%
=====		55%
=====		56%
=====		56%
=====		57%
=====		58%
=====		58%
=====		59%
=====		60%
=====		61%
=====		62%

=====	62%
=====	63%
=====	64%
=====	64%
=====	65%
=====	66%
=====	66%
=====	67%
=====	68%
=====	68%
=====	69%
=====	70%
=====	71%
=====	72%
=====	72%
=====	73%
=====	74%
=====	74%
=====	75%
=====	76%
=====	76%
=====	77%

=====		78%
=====		78%
=====		79%
=====		80%
=====		81%
=====		82%
=====		82%
=====		83%
=====		84%
=====		84%
=====		85%
=====		86%
=====		86%
=====		87%
=====		88%
=====		88%
=====		89%
=====		90%
=====		91%
=====		92%
=====		92%

=====	93%
=====	94%
=====	94%
=====	95%
=====	96%
=====	96%
=====	97%
=====	98%
=====	98%
=====	99%
=====	100%

```
res_df5 <- r2_indirect(fit5, x_var = "X", y_var = "Y", Med_Path = c("a", "b"),
n_boot = 200)
```

Running Bootstrap on 1 specific path(s) + Total Mediated Effect...

		0%
=		1%
=		2%
==		2%
==		3%
==		4%
===		4%

=====	5%
=====	6%
=====	6%
=====	7%
=====	8%
=====	8%
=====	9%
=====	10%
=====	11%
=====	12%
=====	12%
=====	13%
=====	14%
=====	14%
=====	15%
=====	16%
=====	16%
=====	17%
=====	18%
=====	18%
=====	19%
=====	20%

=====		21%
=====		22%
=====		22%
=====		23%
=====		24%
=====		24%
=====		25%
=====		26%
=====		26%
=====		27%
=====		28%
=====		28%
=====		29%
=====		30%
=====		31%
=====		32%
=====		32%
=====		33%
=====		34%
=====		34%
=====		35%

=====	36%
=====	36%
=====	37%
=====	38%
=====	38%
=====	39%
=====	40%
=====	41%
=====	42%
=====	42%
=====	43%
=====	44%
=====	44%
=====	45%
=====	46%
=====	46%
=====	47%
=====	48%
=====	48%
=====	49%
=====	50%
=====	51%

=====		52%
=====		52%
=====		53%
=====		54%
=====		54%
=====		55%
=====		56%
=====		56%
=====		57%
=====		58%
=====		58%
=====		59%
=====		60%
=====		61%
=====		62%
=====		62%
=====		63%
=====		64%
=====		64%
=====		65%
=====		66%

=====	66%
=====	67%
=====	68%
=====	68%
=====	69%
=====	70%
=====	71%
=====	72%
=====	72%
=====	73%
=====	74%
=====	74%
=====	75%
=====	76%
=====	76%
=====	77%
=====	78%
=====	78%
=====	79%
=====	80%
=====	81%
=====	82%

=====		82%
=====		83%
=====		84%
=====		84%
=====		85%
=====		86%
=====		86%
=====		87%
=====		88%
=====		88%
=====		89%
=====		90%
=====		91%
=====		92%
=====		92%
=====		93%
=====		94%
=====		94%
=====		95%
=====		96%
=====		96%

```

===== | 97%
|
===== | 98%
|
===== | 98%
|
===== | 99%
|
===== | 100%

```

```
cat("\n--> FUNCTION REPORT: DATASET 1 (N = 400)\n")
```

```
--> FUNCTION REPORT: DATASET 1 (N = 400)
```

```
print(res_df1, row.names = FALSE)
```

	Effect	r2_indirect	CI_Low	CI_High	Perc	CI_Low_P	CI_High_P
Med_Path [a*b]		0.03925	0.02089	0.06861	3.93%	2.09%	6.86%
Total Mediated Effect		0.03925	0.02089	0.06861	3.93%	2.09%	6.86%

```
cat("\n--> FUNCTION REPORT: DATASET 2 (N = 500)\n")
```

```
--> FUNCTION REPORT: DATASET 2 (N = 500)
```

```
print(res_df2, row.names = FALSE)
```

	Effect	r2_indirect	CI_Low	CI_High	Perc	CI_Low_P	CI_High_P
Med_Path [a*b]		0.08671	0.06311	0.11884	8.67%	6.31%	11.88%
Total Mediated Effect		0.08671	0.06311	0.11884	8.67%	6.31%	11.88%

```
cat("\n--> FUNCTION REPORT: DATASET 3 (N = 600)\n")
```

```
--> FUNCTION REPORT: DATASET 3 (N = 600)
```



```
print(res_df3, row.names = FALSE)
```

	Effect	r2_indirect	CI_Low	CI_High	Perc	CI_Low_P	CI_High_P
Med_Path [a*b]		0.05586	0.03505	0.07598	5.59%	3.5%	7.6%
Total Mediated Effect		0.05586	0.03505	0.07598	5.59%	3.5%	7.6%

```
cat("\n--> FUNCTION REPORT: DATASET 4 (TYPE 1 SUPPRESSION, N = 600)\n")
```

```
--> FUNCTION REPORT: DATASET 4 (TYPE 1 SUPPRESSION, N = 600)
```

```
print(res_df4, row.names = FALSE)
```

	Effect	r2_indirect	CI_Low	CI_High	Perc	CI_Low_P	CI_High_P
Med_Path [a*b]		0.10413	0.07618	0.14712	10.41%	7.62%	14.71%
Total Mediated Effect		0.10413	0.07618	0.14712	10.41%	7.62%	14.71%

```
cat("\n--> FUNCTION REPORT: DATASET 4 (TYPE 2 SUPPRESSION, N = 600)\n")
```

```
--> FUNCTION REPORT: DATASET 4 (TYPE 2 SUPPRESSION, N = 600)
```

```
print(res_df5, row.names = FALSE)
```

	Effect	r2_indirect	CI_Low	CI_High	Perc	CI_Low_P	CI_High_P
Med_Path [a*b]		0.08389	0.05506	0.11616	8.39%	5.51%	11.62%
Total Mediated Effect		0.08389	0.05506	0.11616	8.39%	5.51%	11.62%

```
#
=====
# METHODOLOGICAL CHECK: WRIGHT'S DECOMPOSITION FOR SINGLE MEDIATOR (5 BLOCKS)
#
=====

wright_proof_single <- function(model_fit, dataset_name = "Dataset") {

  # 1. Extract raw parameter estimates from the fitted model
```

```

est <- parameterEstimates(model_fit)

a_est <- est$est[est$label == "a"]
b_est <- est$est[est$label == "b"]
cp_est <- est$est[est$label == "cp"]

# Extract residual variances (e_M and e_Y)
var_eM <- est$est[est$op == "~~" & est$lhs == "M" & est$rhs == "M"]
var_eY <- est$est[est$op == "~~" & est$lhs == "Y" & est$rhs == "Y"]

# Extract observed variances from the sample covariance matrix (Fixed X and Y)
cov_mat <- lavInspect(model_fit, "cov.ov")
var_X <- cov_mat["X", "X"]
var_Y <- cov_mat["Y", "Y"]

# 2. Calculate the 5 components in RAW METRIC
B1_Raw <- (cp_est)^2 * var_X
B2_Raw <- (a_est * b_est)^2 * var_X
B3_Raw <- 2 * cp_est * a_est * b_est * var_X
B4_Raw <- (b_est)^2 * var_eM
B5_Raw <- var_eY

Total_Raw <- B1_Raw + B2_Raw + B3_Raw + B4_Raw + B5_Raw

# 3. Calculate PROPORTIONS and PERCENTAGES
P1 <- B1_Raw / var_Y; P2 <- B2_Raw / var_Y; P3 <- B3_Raw / var_Y
P4 <- B4_Raw / var_Y; P5 <- B5_Raw / var_Y

Total_Prop <- P1 + P2 + P3 + P4 + P5

# --- PRINTING THE COMPREHENSIVE MATHEMATICAL CHECK REPORT ---
cat("\n=====\\n")
cat(" WRIGHT DECOMPOSITION PROOF (SINGLE):", dataset_name, "\\n")
cat("=====\\n")
cat(sprintf(" Observed Variance of Y (Denominator) : %10.5f\\n", var_Y))
cat(sprintf(" Implied Variance of Y (Reconstructed): %10.5f\\n", Total_Raw))
cat(sprintf(" Discrepancy (Reconstruction Error) : %10.5f\\n", var_Y -
Total_Raw))
cat("-----\\n")
cat(sprintf(" %-38s | %-10s | %-10s | %-10s\\n", "COMPONENT", "RAW VAR",
"PROPORTION", "PERCENT"))
cat("-----\\n")

```

```

\n")
  cat(sprintf(" 1. Pure Direct Effect (cp^2)          | %10.5f | %10.5f | %8.2f
%%\n", B1_Raw, P1, P1*100))
  cat(sprintf(" 2. Pure Indirect Effect ((ab)^2)      | %10.5f | %10.5f | %8.2f
%%\n", B2_Raw, P2, P2*100))
  cat(sprintf(" 3. Wright Joint Cov (2*cp*a*b)       | %10.5f | %10.5f | %8.2f
%%\n", B3_Raw, P3, P3*100))
  cat(sprintf(" 4. Independent M Variance (b^2*eM)    | %10.5f | %10.5f | %8.2f
%%\n", B4_Raw, P4, P4*100))
  cat(sprintf(" 5. Y Residual/Error Variance (eY)     | %10.5f | %10.5f | %8.2f
%%\n", B5_Raw, P5, P5*100))
  cat("-----\n")
\n")
  cat(sprintf(" %-38s | %10.5f | %10.5f | %8.2f %%\n", "TOTAL CHECK", Total_Raw,
Total_Prop, Total_Prop*100))
  cat("=====\n")
}

#
=====
# EXECUTE PROOF ON THE FITTED MODELS
#
=====
# Assuming models are already estimated in lavaan using raw data:
wright_proof_single(fit1, dataset_name = "Dataset 1 (N = 400)")

```

```

=====
WRIGHT DECOMPOSITION PROOF (SINGLE): Dataset 1 (N = 400)
=====
Observed Variance of Y (Denominator) :    1.62809
Implied Variance of Y (Reconstructed):    1.62809
Discrepancy (Reconstruction Error)   :   -0.00000
-----

```

COMPONENT	RAW VAR	PROPORTION	PERCENT
1. Pure Direct Effect (cp^2)	0.10882	0.06684	6.68 %
2. Pure Indirect Effect ((ab)^2)	0.06391	0.03925	3.93 %
3. Wright Joint Cov (2*cp*a*b)	0.16679	0.10245	10.24 %
4. Independent M Variance (b^2*eM)	0.24108	0.14807	14.81 %
5. Y Residual/Error Variance (eY)	1.04749	0.64339	64.34 %

```

-----

```

TOTAL CHECK		1.62809		1.00000		100.00 %
=====						

```
wright_proof_single(fit2, dataset_name = "Dataset 2 (N = 500)")
```

=====			
WRIGHT DECOMPOSITION PROOF (SINGLE): Dataset 2 (N = 500)			
=====			
Observed Variance of Y (Denominator) :		1.93909	
Implied Variance of Y (Reconstructed):		1.93909	
Discrepancy (Reconstruction Error) :		0.00000	

COMPONENT		RAW VAR	PROPORTION PERCENT

1. Pure Direct Effect (cp^2)		0.10008	0.05161 5.16 %
2. Pure Indirect Effect ((ab)^2)		0.16814	0.08671 8.67 %
3. Wright Joint Cov (2*cp*a*b)		0.25945	0.13380 13.38 %
4. Independent M Variance (b^2*eM)		0.43391	0.22377 22.38 %
5. Y Residual/Error Variance (eY)		0.97752	0.50411 50.41 %

TOTAL CHECK		1.93909	1.00000 100.00 %
=====			

```
wright_proof_single(fit3, dataset_name = "Dataset 3 (N = 600)")
```

=====			
WRIGHT DECOMPOSITION PROOF (SINGLE): Dataset 3 (N = 600)			
=====			
Observed Variance of Y (Denominator) :		2.31910	
Implied Variance of Y (Reconstructed):		2.31910	
Discrepancy (Reconstruction Error) :		0.00000	

COMPONENT		RAW VAR	PROPORTION PERCENT

1. Pure Direct Effect (cp^2)		0.16380	0.07063 7.06 %
2. Pure Indirect Effect ((ab)^2)		0.12954	0.05586 5.59 %
3. Wright Joint Cov (2*cp*a*b)		0.29133	0.12562 12.56 %
4. Independent M Variance (b^2*eM)		0.76356	0.32925 32.92 %
5. Y Residual/Error Variance (eY)		0.97086	0.41864 41.86 %

```
-----
TOTAL CHECK | 2.31910 | 1.00000 | 100.00 %
=====
```

```
wright_proof_single(fit4, dataset_name = "Dataset 4 (TYPE 1 SUPPRESSION, N = 600)")
```

```
=====
WRIGHT DECOMPOSITION PROOF (SINGLE): Dataset 4 (TYPE 1 SUPPRESSION, N = 600)
=====
Observed Variance of Y (Denominator) : 1.55023
Implied Variance of Y (Reconstructed): 1.55023
Discrepancy (Reconstruction Error) : -0.00000
-----
COMPONENT | RAW VAR | PROPORTION | PERCENT
-----
1. Pure Direct Effect (cp^2) | 0.58494 | 0.37732 | 37.73 %
2. Pure Indirect Effect ((ab)^2) | 0.16143 | 0.10413 | 10.41 %
3. Wright Joint Cov (2*cp*a*b) | -0.61458 | -0.39644 | -39.64 %
4. Independent M Variance (b^2*eM) | 0.44758 | 0.28872 | 28.87 %
5. Y Residual/Error Variance (eY) | 0.97086 | 0.62627 | 62.63 %
-----
TOTAL CHECK | 1.55023 | 1.00000 | 100.00 %
=====
```

```
wright_proof_single(fit4, dataset_name = "Dataset 4 (TYPE 2 SUPPRESSION, N = 600)")
```

```
=====
WRIGHT DECOMPOSITION PROOF (SINGLE): Dataset 4 (TYPE 2 SUPPRESSION, N = 600)
=====
Observed Variance of Y (Denominator) : 1.55023
Implied Variance of Y (Reconstructed): 1.55023
Discrepancy (Reconstruction Error) : -0.00000
-----
COMPONENT | RAW VAR | PROPORTION | PERCENT
-----
1. Pure Direct Effect (cp^2) | 0.58494 | 0.37732 | 37.73 %
2. Pure Indirect Effect ((ab)^2) | 0.16143 | 0.10413 | 10.41 %
```

3. Wright Joint Cov ($2*cp*a*b$)		-0.61458		-0.39644		-39.64 %
4. Independent M Variance (b^2*eM)		0.44758		0.28872		28.87 %
5. Y Residual/Error Variance (eY)		0.97086		0.62627		62.63 %

TOTAL CHECK		1.55023		1.00000		100.00 %
=====						

```
#####
```

4.1.2 Models with Two Parallel Mediators

```
#
=====
# DATA GENERATION: TWO PARALLEL MEDIATORS
#
=====

simulate_data_parallel <- function(N, a1, b1, a2, b2, cp, res_cov, seed = 123) {
  set.seed(seed)
  X <- rnorm(N, mean = 0, sd = 1)

  # Inject a shared unobserved variance component (creates residual covariance)
  shared_residual <- rnorm(N, mean = 0, sd = sqrt(res_cov))

  M1 <- a1 * X + shared_residual + rnorm(N, mean = 0, sd = sqrt(0.64 - res_cov))
  M2 <- a2 * X + shared_residual + rnorm(N, mean = 0, sd = sqrt(0.64 - res_cov))

  Y <- cp * X + b1 * M1 + b2 * M2 + rnorm(N, mean = 0, sd = 0.5)

  return(data.frame(X = X, M1 = M1, M2 = M2, Y = Y))
}

# -----
# Creation of the Datasets
# -----

# Dataset 1: N = 600, Standard (All positive cooperative effects)
dfp1 <- simulate_data_parallel(N = 600,
                              a1 = 0.4, b1 = 0.3,
                              a2 = 0.3, b2 = 0.4,
                              cp = 0.3, res_cov = 0.25, seed = 111)

# Dataset 2: N = 700, Standard (All positive cooperative effects)
```

```

dfp2 <- simulate_data_parallel(N = 700,
                              a1 = 0.5, b1 = 0.25,
                              a2 = 0.4, b2 = 0.3,
                              cp = 0.35, res_cov = 0.20, seed = 222)

# Dataset 3: N = 700, Type 1 Suppression (Direct Positive vs. M1 (indirect)
Negative)
dfp3 <- simulate_data_parallel(N = 700,
                              a1 = 0.4, b1 = -0.4,
                              a2 = 0.3, b2 = 0.3,
                              cp = 0.4, res_cov = 0.30, seed = 333)

# Dataset 4: N = 700, Type 2 Suppression (Direct Positive vs. M1 & M2 (indirects)
Negative)
dfp4 <- simulate_data_parallel(N = 700,
                              a1 = 0.4, b1 = -0.4,
                              a2 = -0.3, b2 = 0.4,
                              cp = 0.5, res_cov = 0.28, seed = 444)

# Dataset 5: N = 700, Type 3 Suppression (Direct Negative vs. M1 & M2 (indirects)
Positive)
dfp5 <- simulate_data_parallel(N = 700,
                              a1 = 0.4, b1 = 0.4,
                              a2 = 0.3, b2 = 0.4,
                              cp = -0.5, res_cov = 0.28, seed = 444)

#
=====
# LAVAAN MODEL MODEL DEFINITION (df = 0)
#
=====
# Load the required package
library(lavaan)

parallel_model <- "
M1 ~ a1*X
M2 ~ a2*X
Y ~ cp*X + b1*M1 + b2*M2

#Residual covariance for perfect identification (df=0)
M1 ~~ M2

#Indirect effects

```

```

indM1 := a1*b1
indM2 := a2*b2
"

fitp1 <- sem(parallel_model, estimator = "MLR", data = dfp1)
fitp2 <- sem(parallel_model, estimator = "MLR", data = dfp2)
fitp3 <- sem(parallel_model, estimator = "MLR", data = dfp3)
fitp4 <- sem(parallel_model, estimator = "MLR", data = dfp4)
fitp5 <- sem(parallel_model, estimator = "MLR", data = dfp5)

summary(fitp1, standardized = TRUE)

```

lavaan 0.6-21 ended normally after 8 iterations

Estimator	ML
Optimization method	NLMINB
Number of model parameters	9

Number of observations	600
------------------------	-----

Model Test User Model:

	Standard	Scaled
Test Statistic	0.000	0.000
Degrees of freedom	0	0

Parameter Estimates:

Standard errors	Sandwich
Information bread	Observed
Observed information based on	Hessian

Regressions:

		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
M1 ~							
X	(a1)	0.416	0.032	13.062	0.000	0.416	0.476
M2 ~							
X	(a2)	0.303	0.034	8.971	0.000	0.303	0.352
Y ~							
X	(cp)	0.327	0.027	12.340	0.000	0.327	0.359
M1	(b1)	0.302	0.031	9.807	0.000	0.302	0.290
M2	(b2)	0.406	0.028	14.271	0.000	0.406	0.383

Covariances:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M1 ~~ .M2	0.243	0.028	8.800	0.000	0.243	0.400

Variances:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M1	0.582	0.032	18.037	0.000	0.582	0.773
.M2	0.636	0.041	15.696	0.000	0.636	0.876
.Y	0.274	0.016	17.223	0.000	0.274	0.335

Defined Parameters:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
indM1	0.126	0.016	7.706	0.000	0.126	0.138
indM2	0.123	0.016	7.562	0.000	0.123	0.135

```
summary(fitp2, standardized = TRUE)
```

lavaan 0.6-21 ended normally after 9 iterations

Estimator	ML
Optimization method	NLMINB
Number of model parameters	9
Number of observations	700

Model Test User Model:

	Standard	Scaled
Test Statistic	0.000	0.000
Degrees of freedom	0	0

Parameter Estimates:

Standard errors	Sandwich
Information bread	Observed
Observed information based on	Hessian

Regressions:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
M1 ~ X	(a1) 0.493	0.031	16.147	0.000	0.493	0.527
M2 ~						

X	(a2)	0.368	0.030	12.188	0.000	0.368	0.423
Y ~							
X	(cp)	0.320	0.023	14.131	0.000	0.320	0.390
M1	(b1)	0.309	0.024	13.081	0.000	0.309	0.351
M2	(b2)	0.236	0.025	9.357	0.000	0.236	0.250
Covariances:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M1 ~~							
.M2		0.193	0.023	8.306	0.000	0.193	0.316
Variances:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M1		0.614	0.034	17.873	0.000	0.614	0.722
.M2		0.607	0.031	19.735	0.000	0.607	0.821
.Y		0.232	0.012	18.973	0.000	0.232	0.353
Defined Parameters:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
indM1		0.152	0.015	10.287	0.000	0.152	0.185
indM2		0.087	0.012	7.393	0.000	0.087	0.106

```
summary(fitp3, standardized = TRUE)
```

lavaan 0.6-21 ended normally after 10 iterations

Estimator	ML
Optimization method	NLMINB
Number of model parameters	9

Number of observations	700
------------------------	-----

Model Test User Model:

	Standard	Scaled
Test Statistic	0.000	0.000
Degrees of freedom	0	0

Parameter Estimates:

Standard errors	Sandwich
Information bread	Observed
Observed information based on	Hessian

Regressions:

		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
M1 ~							
X	(a1)	0.406	0.031	13.024	0.000	0.406	0.438
M2 ~							
X	(a2)	0.323	0.031	10.410	0.000	0.323	0.371
Y ~							
X	(cp)	0.398	0.024	16.718	0.000	0.398	0.587
M1	(b1)	-0.355	0.026	-13.550	0.000	-0.355	-0.486
M2	(b2)	0.271	0.027	9.932	0.000	0.271	0.349

Covariances:

		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M1 ~~							
.M2		0.284	0.027	10.474	0.000	0.284	0.443

Variances:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M1	0.658	0.033	19.781	0.000	0.658	0.808
.M2	0.623	0.033	18.615	0.000	0.623	0.863
.Y	0.251	0.014	18.529	0.000	0.251	0.576

Defined Parameters:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
indM1	-0.144	0.016	-9.158	0.000	-0.144	-0.213
indM2	0.088	0.012	7.067	0.000	0.088	0.129

```
summary(fitp4, standardized = TRUE)
```

lavaan 0.6-21 ended normally after 10 iterations

Estimator	ML
Optimization method	NLMINB
Number of model parameters	9
Number of observations	700

Model Test User Model:

	Standard	Scaled
Test Statistic	0.000	0.000
Degrees of freedom	0	0

Parameter Estimates:

Standard errors	Sandwich
Information bread	Observed
Observed information based on	Hessian

Regressions:

		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
M1 ~							
X	(a1)	0.427	0.032	13.456	0.000	0.427	0.461
M2 ~							
X	(a2)	-0.310	0.030	-10.186	0.000	-0.310	-0.359
Y ~							
X	(cp)	0.501	0.025	19.684	0.000	0.501	0.780
M1	(b1)	-0.401	0.026	-15.337	0.000	-0.401	-0.578
M2	(b2)	0.388	0.026	15.099	0.000	0.388	0.523

Covariances:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M1 ~~						
.M2	0.278	0.028	9.869	0.000	0.278	0.418

Variances:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M1	0.678	0.038	17.767	0.000	0.678	0.787
.M2	0.653	0.037	17.578	0.000	0.653	0.871
.Y	0.249	0.013	18.545	0.000	0.249	0.602

Defined Parameters:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
indM1	-0.171	0.017	-9.878	0.000	-0.171	-0.267
indM2	-0.120	0.014	-8.626	0.000	-0.120	-0.188

```
summary(fitp5, standardized = TRUE)
```

lavaan 0.6-21 ended normally after 10 iterations

Estimator	ML
Optimization method	NLMINB
Number of model parameters	9

Number of observations				700			
Model Test User Model:							
				Standard		Scaled	
Test Statistic				0.000		0.000	
Degrees of freedom				0		0	
Parameter Estimates:							
Standard errors				Sandwich			
Information bread				Observed			
Observed information based on				Hessian			
Regressions:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
M1 ~							
X	(a1)	0.427	0.032	13.456	0.000	0.427	0.461
M2 ~							
X	(a2)	0.290	0.030	9.513	0.000	0.290	0.339
Y ~							
X	(cp)	-0.493	0.023	-21.728	0.000	-0.493	-0.645
M1	(b1)	0.399	0.026	15.279	0.000	0.399	0.484
M2	(b2)	0.388	0.026	15.099	0.000	0.388	0.435
Covariances:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M1 ~~							
.M2		0.278	0.028	9.869	0.000	0.278	0.418
Variances:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M1		0.678	0.038	17.767	0.000	0.678	0.787
.M2		0.653	0.037	17.578	0.000	0.653	0.885
.Y		0.249	0.013	18.545	0.000	0.249	0.425
Defined Parameters:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
indM1		0.170	0.016	10.352	0.000	0.170	0.223
indM2		0.113	0.014	7.890	0.000	0.113	0.147

=====

RUNNING r2_indirect ON RAW COEFFICIENTS (UNSTANDARDIZED b) TWO PARALLEL

MEDIATORS

```
# =====
```

```
cat("\n=== LAUNCHING r2_indirect ON RAW UNSTANDARDIZED DATA ===\n")
```

```
=== LAUNCHING r2_indirect ON RAW UNSTANDARDIZED DATA ===
```

```
res_dfp1 <- r2_indirect(fitp1,  
                        x_var = "X",  
                        y_var = "Y",  
                        Med_Path1 = c("a1", "b1"),  
                        Med_Path2 = c("a2", "b2"),  
                        n_boot = 200)
```

Running Bootstrap on 2 specific path(s) + Total Mediated Effect...

		0%
=		1%
=		2%
==		2%
==		3%
==		4%
===		4%
====		5%
====		6%
=====		6%
=====		7%
=====		8%

=====		8%
=====		9%
=====		10%
=====		11%
=====		12%
=====		12%
=====		13%
=====		14%
=====		14%
=====		15%
=====		16%
=====		16%
=====		17%
=====		18%
=====		18%
=====		19%
=====		20%
=====		21%
=====		22%
=====		22%
=====		23%

=====	24%
=====	24%
=====	25%
=====	26%
=====	26%
=====	27%
=====	28%
=====	28%
=====	29%
=====	30%
=====	31%
=====	32%
=====	32%
=====	33%
=====	34%
=====	34%
=====	35%
=====	36%
=====	36%
=====	37%
=====	38%
=====	38%

=====		39%
=====		40%
=====		41%
=====		42%
=====		42%
=====		43%
=====		44%
=====		44%
=====		45%
=====		46%
=====		46%
=====		47%
=====		48%
=====		48%
=====		49%
=====		50%
=====		51%
=====		52%
=====		52%
=====		53%
=====		54%

=====	54%
=====	55%
=====	56%
=====	56%
=====	57%
=====	58%
=====	58%
=====	59%
=====	60%
=====	61%
=====	62%
=====	62%
=====	63%
=====	64%
=====	64%
=====	65%
=====	66%
=====	66%
=====	67%
=====	68%
=====	68%
=====	69%

	=====	70%
	=====	71%
	=====	72%
	=====	72%
	=====	73%
	=====	74%
	=====	74%
	=====	75%
	=====	76%
	=====	76%
	=====	77%
	=====	78%
	=====	78%
	=====	79%
	=====	80%
	=====	81%
	=====	82%
	=====	82%
	=====	83%
	=====	84%
	=====	84%

=====	85%
=====	86%
=====	86%
=====	87%
=====	88%
=====	88%
=====	89%
=====	90%
=====	91%
=====	92%
=====	92%
=====	93%
=====	94%
=====	94%
=====	95%
=====	96%
=====	96%
=====	97%
=====	98%
=====	98%
=====	99%
=====	100%

```
res_dfp2 <- r2_indirect(fitp2,
  x_var = "X",
  y_var = "Y",
  Med_Path1 = c("a1", "b1"),
  Med_Path2 = c("a2", "b2"),
  n_boot = 200)
```

Running Bootstrap on 2 specific path(s) + Total Mediated Effect...

		0%
=		1%
=		2%
==		2%
==		3%
==		4%
===		4%
====		5%
====		6%
=====		6%
=====		7%
=====		8%
=====		8%
=====		9%
=====		10%
=====		11%

=====	12%
=====	12%
=====	13%
=====	14%
=====	14%
=====	15%
=====	16%
=====	16%
=====	17%
=====	18%
=====	18%
=====	19%
=====	20%
=====	21%
=====	22%
=====	22%
=====	23%
=====	24%
=====	24%
=====	25%
=====	26%
=====	26%

=====		27%
=====		28%
=====		28%
=====		29%
=====		30%
=====		31%
=====		32%
=====		32%
=====		33%
=====		34%
=====		34%
=====		35%
=====		36%
=====		36%
=====		37%
=====		38%
=====		38%
=====		39%
=====		40%
=====		41%
=====		42%

=====	42%
=====	43%
=====	44%
=====	44%
=====	45%
=====	46%
=====	46%
=====	47%
=====	48%
=====	48%
=====	49%
=====	50%
=====	51%
=====	52%
=====	52%
=====	53%
=====	54%
=====	54%
=====	55%
=====	56%
=====	56%
=====	57%

	=====	58%
	=====	58%
	=====	59%
	=====	60%
	=====	61%
	=====	62%
	=====	62%
	=====	63%
	=====	64%
	=====	64%
	=====	65%
	=====	66%
	=====	66%
	=====	67%
	=====	68%
	=====	68%
	=====	69%
	=====	70%
	=====	71%
	=====	72%
	=====	72%

=====	73%
=====	74%
=====	74%
=====	75%
=====	76%
=====	76%
=====	77%
=====	78%
=====	78%
=====	79%
=====	80%
=====	81%
=====	82%
=====	82%
=====	83%
=====	84%
=====	84%
=====	85%
=====	86%
=====	86%
=====	87%
=====	88%

=====		88%
=====		89%
=====		90%
=====		91%
=====		92%
=====		92%
=====		93%
=====		94%
=====		94%
=====		95%
=====		96%
=====		96%
=====		97%
=====		98%
=====		98%
=====		99%
=====		100%

```
res_dfp3 <- r2_indirect(fitp3,
  x_var = "X",
  y_var = "Y",
  Med_Path1 = c("a1", "b1"),
  Med_Path2 = c("a2", "b2"),
  n_boot = 200)
```

Running Bootstrap on 2 specific path(s) + Total Mediated Effect...

		0%
=		1%
=		2%
==		2%
==		3%
==		4%
===		4%
====		5%
====		6%
=====		6%
=====		7%
=====		8%
=====		8%
=====		9%
=====		10%
=====		11%
=====		12%
=====		12%
=====		13%
=====		14%

=====	14%
=====	15%
=====	16%
=====	16%
=====	17%
=====	18%
=====	18%
=====	19%
=====	20%
=====	21%
=====	22%
=====	22%
=====	23%
=====	24%
=====	24%
=====	25%
=====	26%
=====	26%
=====	27%
=====	28%
=====	28%
=====	29%

=====		30%
=====		31%
=====		32%
=====		32%
=====		33%
=====		34%
=====		34%
=====		35%
=====		36%
=====		36%
=====		37%
=====		38%
=====		38%
=====		39%
=====		40%
=====		41%
=====		42%
=====		42%
=====		43%
=====		44%
=====		44%

=====	45%
=====	46%
=====	46%
=====	47%
=====	48%
=====	48%
=====	49%
=====	50%
=====	51%
=====	52%
=====	52%
=====	53%
=====	54%
=====	54%
=====	55%
=====	56%
=====	56%
=====	57%
=====	58%
=====	58%
=====	59%
=====	60%

=====		61%
=====		62%
=====		62%
=====		63%
=====		64%
=====		64%
=====		65%
=====		66%
=====		66%
=====		67%
=====		68%
=====		68%
=====		69%
=====		70%
=====		71%
=====		72%
=====		72%
=====		73%
=====		74%
=====		74%
=====		75%

=====	76%
=====	76%
=====	77%
=====	78%
=====	78%
=====	79%
=====	80%
=====	81%
=====	82%
=====	82%
=====	83%
=====	84%
=====	84%
=====	85%
=====	86%
=====	86%
=====	87%
=====	88%
=====	88%
=====	89%
=====	90%
=====	91%

=====		92%
=====		92%
=====		93%
=====		94%
=====		94%
=====		95%
=====		96%
=====		96%
=====		97%
=====		98%
=====		98%
=====		99%
=====		100%

```
res_dfp4 <- r2_indirect(fitp4,
                        x_var = "X",
                        y_var = "Y",
                        Med_Path1 = c("a1", "b1"),
                        Med_Path2 = c("a2", "b2"),
                        n_boot = 200)
```

Running Bootstrap on 2 specific path(s) + Total Mediated Effect...

		0%
=		1%
=		2%

==		2%
==		3%
==		4%
===		4%
====		5%
====		6%
=====		6%
=====		7%
=====		8%
=====		8%
=====		9%
=====		10%
=====		11%
=====		12%
=====		12%
=====		13%
=====		14%
=====		14%
=====		15%
=====		16%
=====		16%

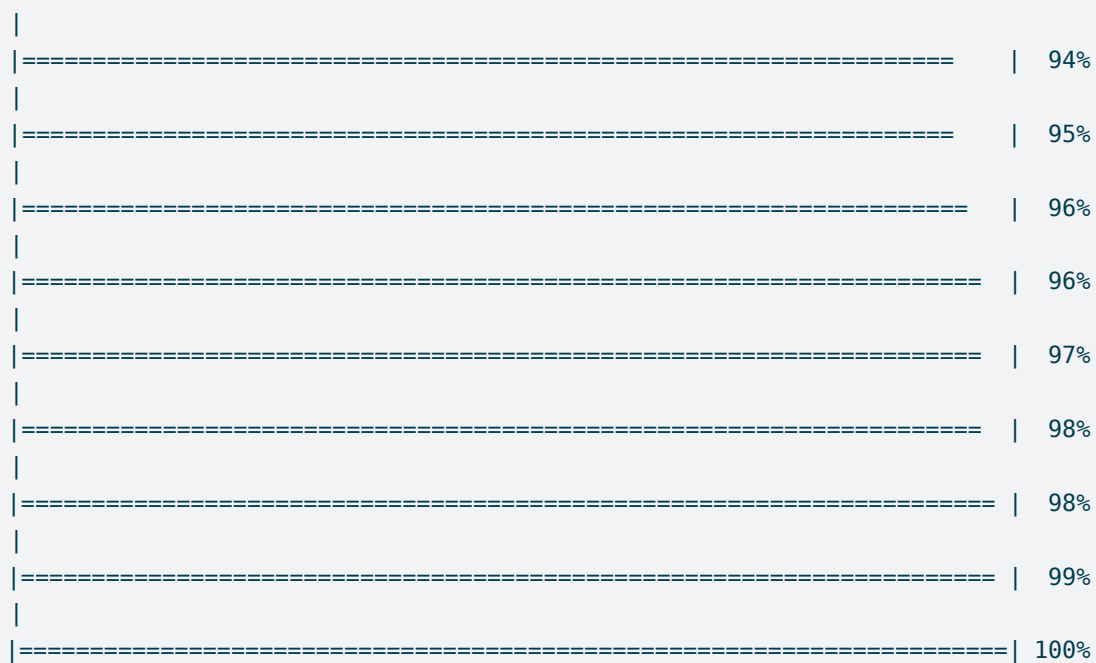
=====	17%
=====	18%
=====	18%
=====	19%
=====	20%
=====	21%
=====	22%
=====	22%
=====	23%
=====	24%
=====	24%
=====	25%
=====	26%
=====	26%
=====	27%
=====	28%
=====	28%
=====	29%
=====	30%
=====	31%
=====	32%
=====	32%

=====		33%
=====		34%
=====		34%
=====		35%
=====		36%
=====		36%
=====		37%
=====		38%
=====		38%
=====		39%
=====		40%
=====		41%
=====		42%
=====		42%
=====		43%
=====		44%
=====		44%
=====		45%
=====		46%
=====		46%
=====		47%

=====	48%
=====	48%
=====	49%
=====	50%
=====	51%
=====	52%
=====	52%
=====	53%
=====	54%
=====	54%
=====	55%
=====	56%
=====	56%
=====	57%
=====	58%
=====	58%
=====	59%
=====	60%
=====	61%
=====	62%
=====	62%
=====	63%

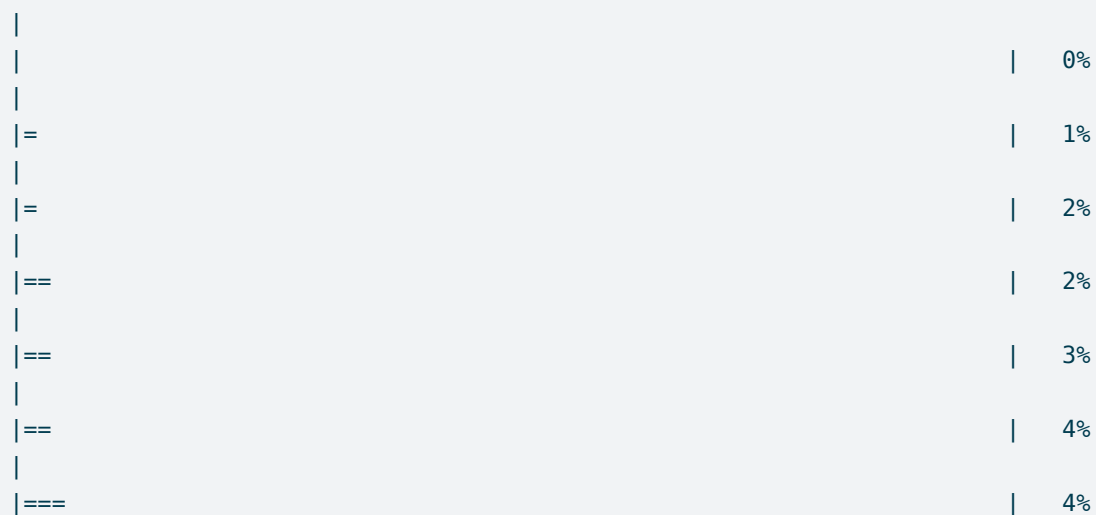
=====		64%
=====		64%
=====		65%
=====		66%
=====		66%
=====		67%
=====		68%
=====		68%
=====		69%
=====		70%
=====		71%
=====		72%
=====		72%
=====		73%
=====		74%
=====		74%
=====		75%
=====		76%
=====		76%
=====		77%
=====		78%

=====	78%
=====	79%
=====	80%
=====	81%
=====	82%
=====	82%
=====	83%
=====	84%
=====	84%
=====	85%
=====	86%
=====	86%
=====	87%
=====	88%
=====	88%
=====	89%
=====	90%
=====	91%
=====	92%
=====	92%
=====	93%
=====	94%



```
res_dfp5 <- r2_indirect(fitp5,
  x_var = "X",
  y_var = "Y",
  Med_Path1 = c("a1", "b1"),
  Med_Path2 = c("a2", "b2"),
  n_boot = 200)
```

Running Bootstrap on 2 specific path(s) + Total Mediated Effect...



=====		5%
=====		6%
=====		6%
=====		7%
=====		8%
=====		8%
=====		9%
=====		10%
=====		11%
=====		12%
=====		12%
=====		13%
=====		14%
=====		14%
=====		15%
=====		16%
=====		16%
=====		17%
=====		18%
=====		18%
=====		19%

=====	20%
=====	21%
=====	22%
=====	22%
=====	23%
=====	24%
=====	24%
=====	25%
=====	26%
=====	26%
=====	27%
=====	28%
=====	28%
=====	29%
=====	30%
=====	31%
=====	32%
=====	32%
=====	33%
=====	34%
=====	34%
=====	35%

=====		36%
=====		36%
=====		37%
=====		38%
=====		38%
=====		39%
=====		40%
=====		41%
=====		42%
=====		42%
=====		43%
=====		44%
=====		44%
=====		45%
=====		46%
=====		46%
=====		47%
=====		48%
=====		48%
=====		49%
=====		50%

=====	51%
=====	52%
=====	52%
=====	53%
=====	54%
=====	54%
=====	55%
=====	56%
=====	56%
=====	57%
=====	58%
=====	58%
=====	59%
=====	60%
=====	61%
=====	62%
=====	62%
=====	63%
=====	64%
=====	64%
=====	65%
=====	66%

	=====	66%
	=====	67%
	=====	68%
	=====	68%
	=====	69%
	=====	70%
	=====	71%
	=====	72%
	=====	72%
	=====	73%
	=====	74%
	=====	74%
	=====	75%
	=====	76%
	=====	76%
	=====	77%
	=====	78%
	=====	78%
	=====	79%
	=====	80%
	=====	81%

=====	82%
=====	82%
=====	83%
=====	84%
=====	84%
=====	85%
=====	86%
=====	86%
=====	87%
=====	88%
=====	88%
=====	89%
=====	90%
=====	91%
=====	92%
=====	92%
=====	93%
=====	94%
=====	94%
=====	95%
=====	96%
=====	96%

```

|
|=====| 97%
|
|=====| 98%
|
|=====| 98%
|
|=====| 99%
|
|=====| 100%

```

```
cat("\n--> FUNCTION REPORT: DATASET 1 (STANDARD, N = 600)\n")
```

```
--> FUNCTION REPORT: DATASET 1 (STANDARD, N = 600)
```

```
print(res_dfp1, row.names = FALSE)
```

	Effect	r2_indirect	CI_Low	CI_High	Perc	CI_Low_P	CI_High_P
Med_Path1	[a1*b1]	0.01905	0.01277	0.02759	1.91%	1.28%	2.76%
Med_Path2	[a2*b2]	0.01818	0.01079	0.02605	1.82%	1.08%	2.6%
Total Mediated Effect		0.07445	0.05644	0.09671	7.45%	5.64%	9.67%

```
cat("\n--> FUNCTION REPORT: DATASET 2 (STANDARD, N = 700)\n")
```

```
--> FUNCTION REPORT: DATASET 2 (STANDARD, N = 700)
```

```
print(res_dfp2, row.names = FALSE)
```

	Effect	r2_indirect	CI_Low	CI_High	Perc	CI_Low_P	CI_High_P
Med_Path1	[a1*b1]	0.03431	0.02341	0.04975	3.43%	2.34%	4.97%
Med_Path2	[a2*b2]	0.01118	0.00648	0.01709	1.12%	0.65%	1.71%
Total Mediated Effect		0.08466	0.06468	0.11159	8.47%	6.47%	11.16%

```
cat("\n--> FUNCTION REPORT: DATASET 3 (TYPE 1 SUPPRESSION, N = 700)\n")
```



```
--> FUNCTION REPORT: DATASET 3 (TYPE 1 SUPPRESSION, N = 700)
```

```
print(res_dfp3, row.names = FALSE)
```

	Effect	r2_indirect	CI_Low	CI_High	Perc	CI_Low_P	CI_High_P
Med_Path1	[a1*b1]	0.04532	0.02801	0.07274	4.53%	2.8%	7.27%
Med_Path2	[a2*b2]	0.01674	0.00963	0.02624	1.67%	0.96%	2.62%
Total Mediated Effect		0.00697	0.00187	0.01509	0.7%	0.19%	1.51%

```
cat("\n--> FUNCTION REPORT: DATASET 4 (TYPE 2 SUPPRESSION, N = 700)\n")
```

```
--> FUNCTION REPORT: DATASET 4 (TYPE 2 SUPPRESSION, N = 700)
```

```
print(res_dfp4, row.names = FALSE)
```

	Effect	r2_indirect	CI_Low	CI_High	Perc	CI_Low_P	CI_High_P
Med_Path1	[a1*b1]	0.07109	0.04789	0.09615	7.11%	4.79%	9.61%
Med_Path2	[a2*b2]	0.03529	0.02330	0.04933	3.53%	2.33%	4.93%
Total Mediated Effect		0.20655	0.15416	0.25631	20.66%	15.42%	25.63%

```
cat("\n--> FUNCTION REPORT: DATASET 5 (TYPE 3 SUPPRESSION, N = 700)\n")
```

```
--> FUNCTION REPORT: DATASET 5 (TYPE 3 SUPPRESSION, N = 700)
```

```
print(res_dfp5, row.names = FALSE)
```

	Effect	r2_indirect	CI_Low	CI_High	Perc	CI_Low_P	CI_High_P
Med_Path1	[a1*b1]	0.04981	0.03276	0.07102	4.98%	3.28%	7.1%
Med_Path2	[a2*b2]	0.02173	0.01307	0.03253	2.17%	1.31%	3.25%
Total Mediated Effect		0.13734	0.10069	0.18186	13.73%	10.07%	18.19%

```
# =====  
# METHODOLOGICAL CHECK: WRIGHT'S DECOMPOSITION FOR PARALLEL MEDIATORS (10 BLOCKS)  
# =====
```

```

wright_proof_parallel <- function(model_fit, dataset_name = "Dataset") {

  est <- parameterEstimates(model_fit)

  # Regressions
  a1_est <- est$est[est$label == "a1"]
  b1_est <- est$est[est$label == "b1"]
  a2_est <- est$est[est$label == "a2"]
  b2_est <- est$est[est$label == "b2"]
  cp_est <- est$est[est$label == "cp"]

  # Residual Variances
  var_eM1 <- est$est[est$op == "~~" & est$lhs == "M1" & est$rhs == "M1"]
  var_eM2 <- est$est[est$op == "~~" & est$lhs == "M2" & est$rhs == "M2"]
  var_eY <- est$est[est$op == "~~" & est$lhs == "Y" & est$rhs == "Y"]

  # Residual Covariance of parallel mediators
  cov_eM12_est <- est$est[est$op == "~~" &
    ((est$lhs == "M1" & est$rhs == "M2") |
     (est$lhs == "M2" & est$rhs == "M1"))]

  # Matrix Sample Variances
  cov_mat <- lavInspect(model_fit, "cov.ov")
  var_X <- cov_mat["X", "X"]
  var_Y <- cov_mat["Y", "Y"]

  # Compute the 10 Raw Components
  B1_Raw <- (cp_est)^2 * var_X
  B2_Raw <- (a1_est * b1_est)^2 * var_X
  B3_Raw <- (a2_est * b2_est)^2 * var_X
  B4_Raw <- 2 * cp_est * a1_est * b1_est * var_X
  B5_Raw <- 2 * cp_est * a2_est * b2_est * var_X
  B6_Raw <- 2 * (a1_est * b1_est) * (a2_est * b2_est) * var_X
  B7_Raw <- (b1_est)^2 * var_eM1
  B8_Raw <- (b2_est)^2 * var_eM2
  B9_Raw <- 2 * b1_est * b2_est * cov_eM12_est
  B10_Raw <- var_eY

  Total_Raw <- B1_Raw + B2_Raw + B3_Raw + B4_Raw + B5_Raw + B6_Raw + B7_Raw +
  B8_Raw + B9_Raw + B10_Raw

  # Proportions
  P1 <- B1_Raw / var_Y; P2 <- B2_Raw / var_Y; P3 <- B3_Raw / var_Y

```

```

P4 <- B4_Raw / var_Y; P5 <- B5_Raw / var_Y; P6 <- B6_Raw / var_Y
P7 <- B7_Raw / var_Y; P8 <- B8_Raw / var_Y; P9 <- B9_Raw / var_Y; P10 <-
B10_Raw / var_Y

Total_Prop <- P1 + P2 + P3 + P4 + P5 + P6 + P7 + P8 + P9 + P10

# --- PRINTING THE COMPREHENSIVE MATHEMATICAL CHECK REPORT ---
cat("\n=====\\n")
cat(" WRIGHT DECOMPOSITION PROOF (PARALLEL):", dataset_name, "\\n")
cat("=====\\n")
cat(sprintf(" Observed Variance of Y (Denominator) : %10.5f\\n", var_Y))
cat(sprintf(" Implied Variance of Y (Reconstructed): %10.5f\\n", Total_Raw))
cat(sprintf(" Discrepancy (Reconstruction Error)      : %10.5f\\n", var_Y -
Total_Raw))
cat("-----\\n")
cat(sprintf(" %-38s | %-10s | %-10s | %-10s\\n", "COMPONENT", "RAW VAR",
"PROPORTION", "PERCENT"))
cat("-----\\n")
cat(sprintf(" 1. Pure Direct Effect (cp^2) | %10.5f | %10.5f | %8.2f
%%\\n", B1_Raw, P1, P1*100))
cat(sprintf(" 2. Pure Indirect M1 ((a1*b1)^2) | %10.5f | %10.5f | %8.2f
%%\\n", B2_Raw, P2, P2*100))
cat(sprintf(" 3. Pure Indirect M2 ((a2*b2)^2) | %10.5f | %10.5f | %8.2f
%%\\n", B3_Raw, P3, P3*100))
cat(sprintf(" 4. Wright Joint Cov (2*cp*a1*b1) | %10.5f | %10.5f | %8.2f
%%\\n", B4_Raw, P4, P4*100))
cat(sprintf(" 5. Wright Joint Cov (2*cp*a2*b2) | %10.5f | %10.5f | %8.2f
%%\\n", B5_Raw, P5, P5*100))
cat(sprintf(" 6. Wright Joint Cov (2*a1*b1*a2*b2) | %10.5f | %10.5f | %8.2f
%%\\n", B6_Raw, P6, P6*100))
cat(sprintf(" 7. Independent M1 Error (b1^2*eM1) | %10.5f | %10.5f | %8.2f
%%\\n", B7_Raw, P7, P7*100))
cat(sprintf(" 8. Independent M2 Error (b2^2*eM2) | %10.5f | %10.5f | %8.2f
%%\\n", B8_Raw, P8, P8*100))
cat(sprintf(" 9. Residual Joint Cov (2*b1*b2*cov) | %10.5f | %10.5f | %8.2f
%%\\n", B9_Raw, P9, P9*100))
cat(sprintf(" 10. Y Residual/Error Var (eY) | %10.5f | %10.5f | %8.2f
%%\\n", B10_Raw, P10, P10*100))
cat("-----\\n")
cat(sprintf(" %-38s | %10.5f | %10.5f | %8.2f %%\\n", "TOTAL CHECK", Total_Raw,

```

```
Total_Prop, Total_Prop*100))
  cat("=====\n")
}

#
=====
# EXECUTE PROOF ON THE FITTED MODELS
#
=====
# Assuming models are already estimated in lavaan using raw data:
wright_proof_parallel(fitp1, "Dataset 1 (N=600, Standard)")
```

```
=====
WRIGHT DECOMPOSITION PROOF (PARALLEL): Dataset 1 (N=600, Standard)
=====
Observed Variance of Y (Denominator) :    0.81921
Implied Variance of Y (Reconstructed):    0.81921
Discrepancy (Reconstruction Error)   :    0.00000
-----
```

COMPONENT	RAW VAR	PROPORTION	PERCENT
1. Pure Direct Effect (cp^2)	0.10540	0.12867	12.87 %
2. Pure Indirect M1 ((a1*b1)^2)	0.01561	0.01905	1.91 %
3. Pure Indirect M2 ((a2*b2)^2)	0.01489	0.01818	1.82 %
4. Wright Joint Cov (2*cp*a1*b1)	0.08112	0.09902	9.90 %
5. Wright Joint Cov (2*cp*a2*b2)	0.07924	0.09673	9.67 %
6. Wright Joint Cov (2*a1*b1*a2*b2)	0.03049	0.03722	3.72 %
7. Independent M1 Error (b1^2*eM1)	0.05322	0.06497	6.50 %
8. Independent M2 Error (b2^2*eM2)	0.10513	0.12833	12.83 %
9. Residual Joint Cov (2*b1*b2*cov)	0.05987	0.07308	7.31 %
10. Y Residual/Error Var (eY)	0.27422	0.33474	33.47 %
TOTAL CHECK	0.81921	1.00000	100.00 %

```
=====
```

```
wright_proof_parallel(fitp2, "Dataset 2 (N=700, Standard)")
```

```
=====
WRIGHT DECOMPOSITION PROOF (PARALLEL): Dataset 2 (N=700, Standard)
```

Observed Variance of Y (Denominator) :		0.65705	
Implied Variance of Y (Reconstructed):		0.65705	
Discrepancy (Reconstruction Error) :		0.00000	

COMPONENT	RAW VAR	PROPORTION	PERCENT

1. Pure Direct Effect (cp^2)	0.09987	0.15201	15.20 %
2. Pure Indirect M1 ((a1*b1)^2)	0.02255	0.03431	3.43 %
3. Pure Indirect M2 ((a2*b2)^2)	0.00735	0.01118	1.12 %
4. Wright Joint Cov (2*cp*a1*b1)	0.09490	0.14444	14.44 %
5. Wright Joint Cov (2*cp*a2*b2)	0.05417	0.08245	8.24 %
6. Wright Joint Cov (2*a1*b1*a2*b2)	0.02574	0.03917	3.92 %
7. Independent M1 Error (b1^2*eM1)	0.05853	0.08909	8.91 %
8. Independent M2 Error (b2^2*eM2)	0.03374	0.05135	5.14 %
9. Residual Joint Cov (2*b1*b2*cov)	0.02805	0.04269	4.27 %
10. Y Residual/Error Var (eY)	0.23215	0.35332	35.33 %

TOTAL CHECK	0.65705	1.00000	100.00 %
=====			

wright_proof_parallel(fitp3, "Dataset 3 (N=700, Type 1 Suppression)")

WRIGHT DECOMPOSITION PROOF (PARALLEL): Dataset 3 (N=700, Type 1 Suppression)

Observed Variance of Y (Denominator) :0.43561

Implied Variance of Y (Reconstructed):0.43561

Discrepancy (Reconstruction Error) : -0.00000

COMPONENT	RAW VAR	PROPORTION	PERCENT
1. Pure Direct Effect (cp^2)	0.15019	0.34478	34.48 %
2. Pure Indirect M1 ($(a1*b1)^2$)	0.01974	0.04532	4.53 %
3. Pure Indirect M2 ($(a2*b2)^2$)	0.00729	0.01674	1.67 %
4. Wright Joint Cov ($2*cp*a1*b1$)	-0.10891	-0.25002	-25.00 %
5. Wright Joint Cov ($2*cp*a2*b2$)	0.06619	0.15196	15.20 %
6. Wright Joint Cov ($2*a1*b1*a2*b2$)	-0.02400	-0.05509	-5.51 %
7. Independent M1 Error ($b1^2*eM1$)	0.08300	0.19053	19.05 %
8. Independent M2 Error ($b2^2*eM2$)	0.04582	0.10519	10.52 %
9. Residual Joint Cov ($2*b1*b2*cov$)	-0.05466	-0.12548	-12.55 %

10. Y Residual/Error Var (eY)		0.25093		0.57606		57.61 %

TOTAL CHECK		0.43561		1.00000		100.00 %
=====						

```
wright_proof_parallel(fitp4, "Dataset 4 (N=700, Type 2 Suppression)")
```

=====			
WRIGHT DECOMPOSITION PROOF (PARALLEL): Dataset 4 (N=700, Type 2 Suppression)			
=====			
Observed Variance of Y (Denominator) :		0.41389	
Implied Variance of Y (Reconstructed):		0.41389	
Discrepancy (Reconstruction Error) :		0.00000	

COMPONENT		RAW VAR	PROPORTION PERCENT

1. Pure Direct Effect (cp^2)		0.25202	0.60891 60.89 %
2. Pure Indirect M1 ((a1*b1)^2)		0.02943	0.07109 7.11 %
3. Pure Indirect M2 ((a2*b2)^2)		0.01460	0.03529 3.53 %
4. Wright Joint Cov (2*cp*a1*b1)		-0.17223	-0.41613 -41.61 %
5. Wright Joint Cov (2*cp*a2*b2)		-0.12134	-0.29316 -29.32 %
6. Wright Joint Cov (2*a1*b1*a2*b2)		0.04146	0.10017 10.02 %
7. Independent M1 Error (b1^2*eM1)		0.10891	0.26314 26.31 %
8. Independent M2 Error (b2^2*eM2)		0.09841	0.23777 23.78 %
9. Residual Joint Cov (2*b1*b2*cov)		-0.08665	-0.20935 -20.94 %
10. Y Residual/Error Var (eY)		0.24927	0.60226 60.23 %

TOTAL CHECK		0.41389	1.00000 100.00 %
=====			

```
wright_proof_parallel(fitp5, "Dataset 5 (N=700, Type 3 Suppression)")
```

=====	
WRIGHT DECOMPOSITION PROOF (PARALLEL): Dataset 5 (N=700, Type 3 Suppression)	
=====	
Observed Variance of Y (Denominator) :	0.58631
Implied Variance of Y (Reconstructed):	0.58631
Discrepancy (Reconstruction Error) :	0.00000

COMPONENT	RAW VAR	PROPORTION	PERCENT
1. Pure Direct Effect (cp^2)	0.24404	0.41624	41.62 %
2. Pure Indirect M1 ($(a1*b1)^2$)	0.02921	0.04981	4.98 %
3. Pure Indirect M2 ($(a2*b2)^2$)	0.01274	0.02173	2.17 %
4. Wright Joint Cov ($2*cp*a1*b1$)	-0.16885	-0.28799	-28.80 %
5. Wright Joint Cov ($2*cp*a2*b2$)	-0.11152	-0.19021	-19.02 %
6. Wright Joint Cov ($2*a1*b1*a2*b2$)	0.03858	0.06580	6.58 %
7. Independent M1 Error ($b1^2*eM1$)	0.10810	0.18437	18.44 %
8. Independent M2 Error ($b2^2*eM2$)	0.09841	0.16785	16.79 %
9. Residual Joint Cov ($2*b1*b2*cov$)	0.08633	0.14723	14.72 %
10. Y Residual/Error Var (eY)	0.24927	0.42515	42.52 %
TOTAL CHECK	0.58631	1.00000	100.00 %

```
#####
```

4.1.3 Models with Two Serial Mediators

```
#
=====
# DATA GENERATION: TWO SERIAL MEDIATORS
#
=====

generate_serial_data <- function(N, a1, a2, d21, b1, b2, cp, seed = 17) {
  set.seed(seed)

  # Generating the independent variable X
  X <- rnorm(N, mean = 0, sd = 1)

  # Eq 1: M1 depends only on X
  e_M1 <- rnorm(N, mean = 0, sd = 1)
  M1 <- (a1 * X) + e_M1

  # Eq 2: M2 depends on X and M1 (Serial Path d21)
  e_M2 <- rnorm(N, mean = 0, sd = 1)
  M2 <- (a2 * X) + (d21 * M1) + e_M2

  # Eq 3: Y depends on X, M1 and M2
  e_Y <- rnorm(N, mean = 0, sd = 1)
  Y <- (cp * X) + (b1 * M1) + (b2 * M2) + e_Y
}
```

```

    return(data.frame(X, M1, M2, Y))
}

# -----
# Creation of the Datasets
# -----

# Dataset 1: N = 600, Standard (All positive cooperative effects)
dfs1 <- generate_serial_data(N = 600,
                             a1 = 0.5, a2 = 0.4, d21 = 0.5,
                             b1 = 0.4, b2 = 0.5, cp = 0.3,
                             seed = 101)

# Dataset 2: N = 700, Standard (All positive, unbalanced weights on the serial
path)
dfs2 <- generate_serial_data(N = 700,
                             a1 = 0.6, a2 = 0.2, d21 = 0.6,
                             b1 = 0.2, b2 = 0.5, cp = 0.4,
                             seed = 102)

# Dataset 3: N = 700, Type 1 Suppression (Direct Positive vs. M1 & Serial Negative)
dfs3 <- generate_serial_data(N = 700,
                             a1 = 0.6, a2 = 0.2, d21 = 0.5,
                             b1 = -0.5, b2 = 0.6, cp = 0.6,
                             seed = 103)

# Dataset 4: N = 700, Type 2 Suppression (Direct Positive vs. M2 & Serial Negative)
dfs4 <- generate_serial_data(N = 700,
                             a1 = 0.6, a2 = 0.5, d21 = 0.4,
                             b1 = 0.5, b2 = -0.6, cp = 0.2,
                             seed = 104)

# Dataset 5: N = 700, Type 3 Suppression (Direct, M1, M2, Positive vs. Serial
Negative)
dfs5 <- generate_serial_data(N = 700,
                             a1 = 0.6, a2 = 0.3, d21 = -0.7,
                             b1 = 0.4, b2 = 0.5, cp = 0.5,
                             seed = 105)

# Dataset 5: N = 700, Type 4 Suppression (Direct, M1, M2, Positive vs. Serial
Negative)
dfs6 <- generate_serial_data(N = 700,

```



```

a1 = 0.6, a2 = 0.3, d21 = 0.7,
b1 = 0.4, b2 = 0.5, cp = -0.5,
seed = 105)

#
=====
# LAVAAN MODEL MODEL DEFINITION (df = 0)
#
=====
# Load the required package
library(lavaan)

serial_model <- "
  M1 ~ a1*X
  M2 ~ a2*X + d*M1
  Y ~ cp*X + b1*M1 + b2*M2

  #indirect effect via M1
  ind_M1 := a1 * b1

  #indirect effect via M2
  ind_M2 := a2 * b2

  #serial indirect effect via M1 and M2
  ind_serial := a1 * d * b2
"

fits1 <- sem(serial_model, estimator = "MLR", data = dfs1)
fits2 <- sem(serial_model, estimator = "MLR", data = dfs2)
fits3 <- sem(serial_model, estimator = "MLR", data = dfs3)
fits4 <- sem(serial_model, estimator = "MLR", data = dfs4)
fits5 <- sem(serial_model, estimator = "MLR", data = dfs5)
fits6 <- sem(serial_model, estimator = "MLR", data = dfs6)

summary(fits1, standardized = TRUE)

```

lavaan 0.6-21 ended normally after 1 iteration

Estimator	ML
Optimization method	NLMINB
Number of model parameters	9

Number of observations		600					
Model Test User Model:							
		Standard		Scaled			
Test Statistic		0.000		0.000			
Degrees of freedom		0		0			
Parameter Estimates:							
Standard errors		Sandwich					
Information bread		Observed					
Observed information based on		Hessian					
Regressions:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
M1 ~							
X	(a1)	0.535	0.043	12.525	0.000	0.535	0.462
M2 ~							
X	(a2)	0.422	0.048	8.723	0.000	0.422	0.306
M1	(d)	0.466	0.042	10.993	0.000	0.466	0.391
Y ~							
X	(cp)	0.288	0.050	5.735	0.000	0.288	0.179
M1	(b1)	0.446	0.044	10.127	0.000	0.446	0.321
M2	(b2)	0.478	0.037	12.934	0.000	0.478	0.411
Variances:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M1		0.959	0.054	17.631	0.000	0.959	0.786
.M2		1.117	0.062	17.989	0.000	1.117	0.643
.Y		1.017	0.059	17.142	0.000	1.017	0.432
Defined Parameters:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
ind_M1		0.238	0.030	7.912	0.000	0.238	0.148
ind_M2		0.202	0.027	7.383	0.000	0.202	0.125
ind_serial		0.119	0.018	6.467	0.000	0.119	0.074

summary(fits2, standardized = TRUE)

lavaan 0.6-21 ended normally after 1 iteration

Estimator	ML						
Optimization method	NLMINB						
Number of model parameters	9						
Number of observations	700						
Model Test User Model:							
	Standard			Scaled			
Test Statistic	0.000			0.000			
Degrees of freedom	0			0			
Parameter Estimates:							
Standard errors	Sandwich						
Information bread	Observed						
Observed information based on	Hessian						
Regressions:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
M1 ~							
X	(a1)	0.610	0.039	15.510	0.000	0.610	0.518
M2 ~							
X	(a2)	0.126	0.047	2.685	0.007	0.126	0.095
M1	(d)	0.646	0.040	16.185	0.000	0.646	0.574
Y ~							
X	(cp)	0.451	0.046	9.766	0.000	0.451	0.291
M1	(b1)	0.243	0.044	5.532	0.000	0.243	0.185
M2	(b2)	0.509	0.037	13.802	0.000	0.509	0.436
Variances:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M1		0.959	0.050	19.121	0.000	0.959	0.732
.M2		1.004	0.047	21.150	0.000	1.004	0.605
.Y		0.981	0.054	18.125	0.000	0.981	0.434
Defined Parameters:							
		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
ind_M1		0.148	0.029	5.096	0.000	0.148	0.096
ind_M2		0.064	0.024	2.639	0.008	0.064	0.042
ind_serial		0.201	0.023	8.812	0.000	0.201	0.130

summary(fits3, standardized = TRUE)

lavaan 0.6-21 ended normally after 1 iteration

Estimator	ML
Optimization method	NLMINB
Number of model parameters	9

Number of observations	700
------------------------	-----

Model Test User Model:

	Standard	Scaled
Test Statistic	0.000	0.000
Degrees of freedom	0	0

Parameter Estimates:

Standard errors	Sandwich
Information bread	Observed
Observed information based on	Hessian

Regressions:

		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
M1 ~							
X	(a1)	0.691	0.039	17.765	0.000	0.691	0.558
M2 ~							
X	(a2)	0.212	0.045	4.731	0.000	0.212	0.176
M1	(d)	0.456	0.036	12.655	0.000	0.456	0.469
Y ~							
X	(cp)	0.670	0.046	14.505	0.000	0.670	0.507
M1	(b1)	-0.590	0.037	-16.002	0.000	-0.590	-0.553
M2	(b2)	0.619	0.039	15.821	0.000	0.619	0.564

Variances:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M1	1.010	0.055	18.430	0.000	1.010	0.688
.M2	0.909	0.045	20.116	0.000	0.909	0.657
.Y	0.893	0.045	19.958	0.000	0.893	0.535

Defined Parameters:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
ind_M1	-0.408	0.033	-12.187	0.000	-0.408	-0.309
ind_M2	0.131	0.029	4.580	0.000	0.131	0.099
ind_serial	0.195	0.022	9.026	0.000	0.195	0.148

```
summary(fits4, standardized = TRUE)
```

lavaan 0.6-21 ended normally after 1 iteration

Estimator	ML
Optimization method	NLMINB
Number of model parameters	9
Number of observations	700

Model Test User Model:

	Standard	Scaled
Test Statistic	0.000	0.000
Degrees of freedom	0	0

Parameter Estimates:

Standard errors	Sandwich
Information bread	Observed
Observed information based on	Hessian

Regressions:

		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
M1 ~							
X	(a1)	0.596	0.038	15.844	0.000	0.596	0.531
M2 ~							
X	(a2)	0.440	0.043	10.169	0.000	0.440	0.366
M1	(d)	0.355	0.038	9.344	0.000	0.355	0.331
Y ~							
X	(cp)	0.241	0.049	4.881	0.000	0.241	0.197
M1	(b1)	0.503	0.042	11.961	0.000	0.503	0.462
M2	(b2)	-0.609	0.041	-14.849	0.000	-0.609	-0.600

Variances:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M1	0.897	0.049	18.203	0.000	0.897	0.718
.M2	0.901	0.051	17.747	0.000	0.901	0.628
.Y	1.049	0.055	18.956	0.000	1.049	0.710

Defined Parameters:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
ind_M1	0.300	0.031	9.695	0.000	0.300	0.245

ind_M2	-0.268	0.033	-8.213	0.000	-0.268	-0.219
ind_serial	-0.129	0.018	-7.291	0.000	-0.129	-0.106

```
summary(fits5, standardized = TRUE)
```

```
lavaan 0.6-21 ended normally after 1 iteration
```

Estimator	ML
Optimization method	NLMINB
Number of model parameters	9
Number of observations	700

Model Test User Model:

	Standard	Scaled
Test Statistic	0.000	0.000
Degrees of freedom	0	0

Parameter Estimates:

Standard errors	Sandwich
Information bread	Observed
Observed information based on	Hessian

Regressions:

		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
M1 ~							
X	(a1)	0.615	0.039	15.606	0.000	0.615	0.498
M2 ~							
X	(a2)	0.320	0.045	7.116	0.000	0.320	0.249
M1	(d)	-0.693	0.038	-18.169	0.000	-0.693	-0.664
Y ~							
X	(cp)	0.481	0.046	10.358	0.000	0.481	0.347
M1	(b1)	0.424	0.047	8.996	0.000	0.424	0.377
M2	(b2)	0.507	0.039	12.916	0.000	0.507	0.471

Variances:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M1	1.041	0.052	20.115	0.000	1.041	0.752
.M2	0.996	0.055	17.969	0.000	0.996	0.661
.Y	1.055	0.057	18.414	0.000	1.055	0.604

Defined Parameters:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
ind_M1	0.261	0.034	7.727	0.000	0.261	0.188
ind_M2	0.162	0.027	6.124	0.000	0.162	0.117
ind_serial	-0.216	0.025	-8.603	0.000	-0.216	-0.156

```
summary(fits6, standardized = TRUE)
```

lavaan 0.6-21 ended normally after 1 iteration

Estimator	ML
Optimization method	NLMINB
Number of model parameters	9

Number of observations	700
------------------------	-----

Model Test User Model:

	Standard	Scaled
Test Statistic	0.000	0.000
Degrees of freedom	0	0

Parameter Estimates:

Standard errors	Sandwich
Information bread	Observed
Observed information based on	Hessian

Regressions:

		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
M1 ~							
X	(a1)	0.615	0.039	15.606	0.000	0.615	0.498
M2 ~							
X	(a2)	0.320	0.045	7.116	0.000	0.320	0.214
M1	(d)	0.707	0.038	18.560	0.000	0.707	0.584
Y ~							
X	(cp)	-0.519	0.046	-11.156	0.000	-0.519	-0.355
M1	(b1)	0.414	0.047	8.734	0.000	0.414	0.349
M2	(b2)	0.507	0.039	12.916	0.000	0.507	0.519

Variances:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
.M1	1.041	0.052	20.115	0.000	1.041	0.752

.M2	0.996	0.055	17.969	0.000	0.996	0.489
.Y	1.055	0.057	18.414	0.000	1.055	0.542

Defined Parameters:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
ind_M1	0.254	0.033	7.695	0.000	0.254	0.174
ind_M2	0.162	0.027	6.124	0.000	0.162	0.111
ind_serial	0.221	0.025	8.655	0.000	0.221	0.151

```
# =====
# RUNNING r2_indirect ON RAW COEFFICIENTS (UNSTANDARDIZED b) TWO SERIAL MEDIATORS
# =====

cat("\n=== LAUNCHING r2_indirect ON RAW UNSTANDARDIZED DATA ===\n")
```

```
=== LAUNCHING r2_indirect ON RAW UNSTANDARDIZED DATA ===
```

```
res_dfs1 <- r2_indirect(fits1,
                        x_var = "X",
                        y_var = "Y",
                        Med_Path1 = c("a1", "b1"),
                        Med_Path2 = c("a2", "b2"),
                        Serial_Path = c("a1", "d", "b2"),
                        n_boot = 200)
```

Running Bootstrap on 3 specific path(s) + Total Mediated Effect...

		0%
=		1%
=		2%
==		2%
==		3%
==		4%

===		4%
====		5%
=====		6%
=====		6%
=====		7%
=====		8%
=====		8%
=====		9%
=====		10%
=====		11%
=====		12%
=====		12%
=====		13%
=====		14%
=====		14%
=====		15%
=====		16%
=====		16%
=====		17%
=====		18%
=====		18%
=====		19%

=====		20%
=====		21%
=====		22%
=====		22%
=====		23%
=====		24%
=====		24%
=====		25%
=====		26%
=====		26%
=====		27%
=====		28%
=====		28%
=====		29%
=====		30%
=====		31%
=====		32%
=====		32%
=====		33%
=====		34%
=====		34%

=====	35%
=====	36%
=====	36%
=====	37%
=====	38%
=====	38%
=====	39%
=====	40%
=====	41%
=====	42%
=====	42%
=====	43%
=====	44%
=====	44%
=====	45%
=====	46%
=====	46%
=====	47%
=====	48%
=====	48%
=====	49%
=====	50%

=====		51%
=====		52%
=====		52%
=====		53%
=====		54%
=====		54%
=====		55%
=====		56%
=====		56%
=====		57%
=====		58%
=====		58%
=====		59%
=====		60%
=====		61%
=====		62%
=====		62%
=====		63%
=====		64%
=====		64%
=====		65%

=====	66%
=====	66%
=====	67%
=====	68%
=====	68%
=====	69%
=====	70%
=====	71%
=====	72%
=====	72%
=====	73%
=====	74%
=====	74%
=====	75%
=====	76%
=====	76%
=====	77%
=====	78%
=====	78%
=====	79%
=====	80%
=====	81%

=====		82%
=====		82%
=====		83%
=====		84%
=====		84%
=====		85%
=====		86%
=====		86%
=====		87%
=====		88%
=====		88%
=====		89%
=====		90%
=====		91%
=====		92%
=====		92%
=====		93%
=====		94%
=====		94%
=====		95%
=====		96%

=====	96%
=====	97%
=====	98%
=====	98%
=====	99%
=====	100%

```
res_dfs2 <- r2_indirect(fits2,
                        x_var = "X",
                        y_var = "Y",
                        Med_Path1 = c("a1", "b1"),
                        Med_Path2 = c("a2", "b2"),
                        Serial_Path = c("a1", "d", "b2"),
                        n_boot = 200)
```

Running Bootstrap on 3 specific path(s) + Total Mediated Effect...

		0%
=		1%
=		2%
==		2%
==		3%
==		4%
===		4%
====		5%
====		6%
=====		6%

=====		7%
=====		8%
=====		8%
=====		9%
=====		10%
=====		11%
=====		12%
=====		12%
=====		13%
=====		14%
=====		14%
=====		15%
=====		16%
=====		16%
=====		17%
=====		18%
=====		18%
=====		19%
=====		20%
=====		21%
=====		22%

=====	22%
=====	23%
=====	24%
=====	24%
=====	25%
=====	26%
=====	26%
=====	27%
=====	28%
=====	28%
=====	29%
=====	30%
=====	31%
=====	32%
=====	32%
=====	33%
=====	34%
=====	34%
=====	35%
=====	36%
=====	36%
=====	37%

=====		38%
=====		38%
=====		39%
=====		40%
=====		41%
=====		42%
=====		42%
=====		43%
=====		44%
=====		44%
=====		45%
=====		46%
=====		46%
=====		47%
=====		48%
=====		48%
=====		49%
=====		50%
=====		51%
=====		52%
=====		52%

=====	53%
=====	54%
=====	54%
=====	55%
=====	56%
=====	56%
=====	57%
=====	58%
=====	58%
=====	59%
=====	60%
=====	61%
=====	62%
=====	62%
=====	63%
=====	64%
=====	64%
=====	65%
=====	66%
=====	66%
=====	67%
=====	68%

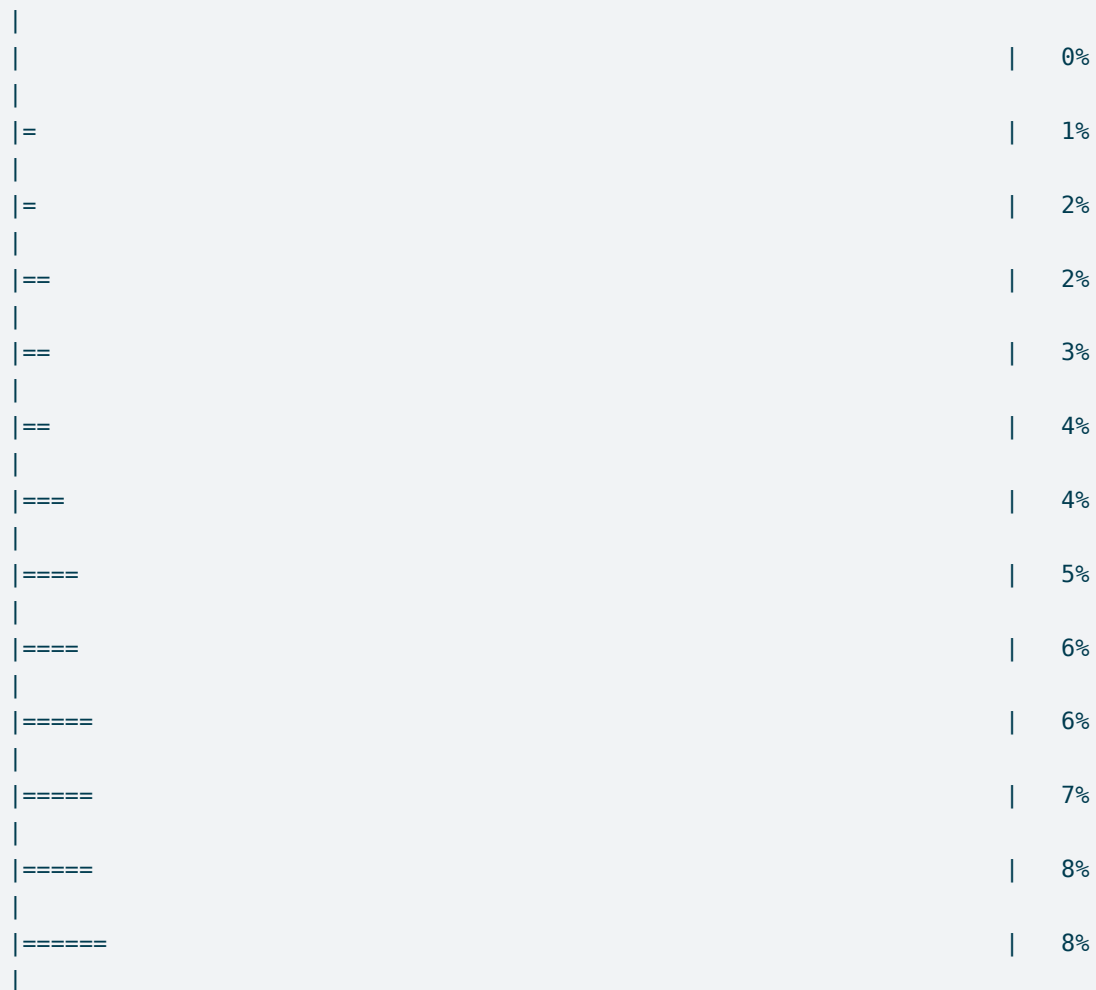
	=====	68%
	=====	69%
	=====	70%
	=====	71%
	=====	72%
	=====	72%
	=====	73%
	=====	74%
	=====	74%
	=====	75%
	=====	76%
	=====	76%
	=====	77%
	=====	78%
	=====	78%
	=====	79%
	=====	80%
	=====	81%
	=====	82%
	=====	82%
	=====	83%

=====	84%
=====	84%
=====	85%
=====	86%
=====	86%
=====	87%
=====	88%
=====	88%
=====	89%
=====	90%
=====	91%
=====	92%
=====	92%
=====	93%
=====	94%
=====	94%
=====	95%
=====	96%
=====	96%
=====	97%
=====	98%
=====	98%



```
res_dfs3 <- r2_indirect(fits3,
                        x_var = "X",
                        y_var = "Y",
                        Med_Path1 = c("a1", "b1"),
                        Med_Path2 = c("a2", "b2"),
                        Serial_Path = c("a1", "d", "b2"),
                        n_boot = 200)
```

Running Bootstrap on 3 specific path(s) + Total Mediated Effect...



=====	9%
=====	10%
=====	11%
=====	12%
=====	12%
=====	13%
=====	14%
=====	14%
=====	15%
=====	16%
=====	16%
=====	17%
=====	18%
=====	18%
=====	19%
=====	20%
=====	21%
=====	22%
=====	22%
=====	23%
=====	24%
=====	24%

=====		25%
=====		26%
=====		26%
=====		27%
=====		28%
=====		28%
=====		29%
=====		30%
=====		31%
=====		32%
=====		32%
=====		33%
=====		34%
=====		34%
=====		35%
=====		36%
=====		36%
=====		37%
=====		38%
=====		38%
=====		39%

=====	40%
=====	41%
=====	42%
=====	42%
=====	43%
=====	44%
=====	44%
=====	45%
=====	46%
=====	46%
=====	47%
=====	48%
=====	48%
=====	49%
=====	50%
=====	51%
=====	52%
=====	52%
=====	53%
=====	54%
=====	54%
=====	55%

	=====	56%
	=====	56%
	=====	57%
	=====	58%
	=====	58%
	=====	59%
	=====	60%
	=====	61%
	=====	62%
	=====	62%
	=====	63%
	=====	64%
	=====	64%
	=====	65%
	=====	66%
	=====	66%
	=====	67%
	=====	68%
	=====	68%
	=====	69%
	=====	70%

=====	71%
=====	72%
=====	72%
=====	73%
=====	74%
=====	74%
=====	75%
=====	76%
=====	76%
=====	77%
=====	78%
=====	78%
=====	79%
=====	80%
=====	81%
=====	82%
=====	82%
=====	83%
=====	84%
=====	84%
=====	85%
=====	86%

=====		86%
=====		87%
=====		88%
=====		88%
=====		89%
=====		90%
=====		91%
=====		92%
=====		92%
=====		93%
=====		94%
=====		94%
=====		95%
=====		96%
=====		96%
=====		97%
=====		98%
=====		98%
=====		99%
=====		100%

```
res_dfs4 <- r2_indirect(fits4,
                        x_var = "X",
                        y_var = "Y",
                        Med_Path1 = c("a1", "b1"),
                        Med_Path2 = c("a2", "b2"),
                        Serial_Path = c("a1", "d", "b2"),
                        n_boot = 200)
```

Running Bootstrap on 3 specific path(s) + Total Mediated Effect...

	0%
=	1%
=	2%
==	2%
==	3%
==	4%
===	4%
====	5%
====	6%
====	6%
====	7%
====	8%
====	8%
====	9%
====	10%
====	11%

=====		12%
=====		12%
=====		13%
=====		14%
=====		14%
=====		15%
=====		16%
=====		16%
=====		17%
=====		18%
=====		18%
=====		19%
=====		20%
=====		21%
=====		22%
=====		22%
=====		23%
=====		24%
=====		24%
=====		25%
=====		26%

=====	26%
=====	27%
=====	28%
=====	28%
=====	29%
=====	30%
=====	31%
=====	32%
=====	32%
=====	33%
=====	34%
=====	34%
=====	35%
=====	36%
=====	36%
=====	37%
=====	38%
=====	38%
=====	39%
=====	40%
=====	41%
=====	42%

=====		42%
=====		43%
=====		44%
=====		44%
=====		45%
=====		46%
=====		46%
=====		47%
=====		48%
=====		48%
=====		49%
=====		50%
=====		51%
=====		52%
=====		52%
=====		53%
=====		54%
=====		54%
=====		55%
=====		56%
=====		56%

=====	57%
=====	58%
=====	58%
=====	59%
=====	60%
=====	61%
=====	62%
=====	62%
=====	63%
=====	64%
=====	64%
=====	65%
=====	66%
=====	66%
=====	67%
=====	68%
=====	68%
=====	69%
=====	70%
=====	71%
=====	72%
=====	72%

=====		73%
=====		74%
=====		74%
=====		75%
=====		76%
=====		76%
=====		77%
=====		78%
=====		78%
=====		79%
=====		80%
=====		81%
=====		82%
=====		82%
=====		83%
=====		84%
=====		84%
=====		85%
=====		86%
=====		86%
=====		87%

=====	88%
=====	88%
=====	89%
=====	90%
=====	91%
=====	92%
=====	92%
=====	93%
=====	94%
=====	94%
=====	95%
=====	96%
=====	96%
=====	97%
=====	98%
=====	98%
=====	99%
=====	100%

```
res_dfs5 <- r2_indirect(fits5,
                        x_var = "X",
                        y_var = "Y",
                        Med_Path1 = c("a1", "b1"),
                        Med_Path2 = c("a2", "b2"),
```

```
Serial_Path = c("a1", "d", "b2"),
n_boot = 200)
```

Running Bootstrap on 3 specific path(s) + Total Mediated Effect...

		0%
=		1%
=		2%
==		2%
==		3%
==		4%
===		4%
====		5%
====		6%
=====		6%
=====		7%
=====		8%
=====		8%
=====		9%
=====		10%
=====		11%
=====		12%
=====		12%

=====	13%
=====	14%
=====	14%
=====	15%
=====	16%
=====	16%
=====	17%
=====	18%
=====	18%
=====	19%
=====	20%
=====	21%
=====	22%
=====	22%
=====	23%
=====	24%
=====	24%
=====	25%
=====	26%
=====	26%
=====	27%
=====	28%

=====		28%
=====		29%
=====		30%
=====		31%
=====		32%
=====		32%
=====		33%
=====		34%
=====		34%
=====		35%
=====		36%
=====		36%
=====		37%
=====		38%
=====		38%
=====		39%
=====		40%
=====		41%
=====		42%
=====		42%
=====		43%

=====	44%
=====	44%
=====	45%
=====	46%
=====	46%
=====	47%
=====	48%
=====	48%
=====	49%
=====	50%
=====	51%
=====	52%
=====	52%
=====	53%
=====	54%
=====	54%
=====	55%
=====	56%
=====	56%
=====	57%
=====	58%
=====	58%

=====		59%
=====		60%
=====		61%
=====		62%
=====		62%
=====		63%
=====		64%
=====		64%
=====		65%
=====		66%
=====		66%
=====		67%
=====		68%
=====		68%
=====		69%
=====		70%
=====		71%
=====		72%
=====		72%
=====		73%
=====		74%

=====	74%
=====	75%
=====	76%
=====	76%
=====	77%
=====	78%
=====	78%
=====	79%
=====	80%
=====	81%
=====	82%
=====	82%
=====	83%
=====	84%
=====	84%
=====	85%
=====	86%
=====	86%
=====	87%
=====	88%
=====	88%
=====	89%

=====		90%
=====		91%
=====		92%
=====		92%
=====		93%
=====		94%
=====		94%
=====		95%
=====		96%
=====		96%
=====		97%
=====		98%
=====		98%
=====		99%
=====		100%

```
res_dfs6 <- r2_indirect(fits6,
  x_var = "X",
  y_var = "Y",
  Med_Path1 = c("a1", "b1"),
  Med_Path2 = c("a2", "b2"),
  Serial_Path = c("a1", "d", "b2"),
  n_boot = 200)
```

Running Bootstrap on 3 specific path(s) + Total Mediated Effect...

|

		0%
=		1%
=		2%
==		2%
==		3%
==		4%
===		4%
====		5%
====		6%
=====		6%
=====		7%
=====		8%
=====		8%
=====		9%
=====		10%
=====		11%
=====		12%
=====		12%
=====		13%
=====		14%
=====		14%
=====		15%

=====		16%
=====		16%
=====		17%
=====		18%
=====		18%
=====		19%
=====		20%
=====		21%
=====		22%
=====		22%
=====		23%
=====		24%
=====		24%
=====		25%
=====		26%
=====		26%
=====		27%
=====		28%
=====		28%
=====		29%
=====		30%

=====	31%
=====	32%
=====	32%
=====	33%
=====	34%
=====	34%
=====	35%
=====	36%
=====	36%
=====	37%
=====	38%
=====	38%
=====	39%
=====	40%
=====	41%
=====	42%
=====	42%
=====	43%
=====	44%
=====	44%
=====	45%
=====	46%

=====		46%
=====		47%
=====		48%
=====		48%
=====		49%
=====		50%
=====		51%
=====		52%
=====		52%
=====		53%
=====		54%
=====		54%
=====		55%
=====		56%
=====		56%
=====		57%
=====		58%
=====		58%
=====		59%
=====		60%
=====		61%

=====	62%
=====	62%
=====	63%
=====	64%
=====	64%
=====	65%
=====	66%
=====	66%
=====	67%
=====	68%
=====	68%
=====	69%
=====	70%
=====	71%
=====	72%
=====	72%
=====	73%
=====	74%
=====	74%
=====	75%
=====	76%
=====	76%

=====		77%
=====		78%
=====		78%
=====		79%
=====		80%
=====		81%
=====		82%
=====		82%
=====		83%
=====		84%
=====		84%
=====		85%
=====		86%
=====		86%
=====		87%
=====		88%
=====		88%
=====		89%
=====		90%
=====		91%
=====		92%


```

===== | 92%
|
===== | 93%
|
===== | 94%
|
===== | 94%
|
===== | 95%
|
===== | 96%
|
===== | 96%
|
===== | 97%
|
===== | 98%
|
===== | 98%
|
===== | 99%
|
===== | 100%

```

```
cat("\n--> FUNCTION REPORT: DATASET 1 (N = 600) STANDARD\n")
```

```
--> FUNCTION REPORT: DATASET 1 (N = 600) STANDARD
```

```
print(res_dfs1, row.names = FALSE)
```

	Effect	r2_indirect	CI_Low	CI_High	Perc	CI_Low_P	CI_High_P
Med_Path1	[a1*b1]	0.02200	0.01389	0.03286	2.2%	1.39%	3.29%
Med_Path2	[a2*b2]	0.01575	0.00915	0.02656	1.57%	0.92%	2.66%
Serial_Path	[a1*d*b2]	0.00551	0.00298	0.01013	0.55%	0.3%	1.01%
Total Mediated Effect		0.12112	0.09645	0.15000	12.11%	9.64%	15%

```
cat("\n--> FUNCTION REPORT: DATASET 2 (N = 700) STANDARD\n")
```

```
--> FUNCTION REPORT: DATASET 2 (N = 700) STANDARD
```

```
print(res_dfs2, row.names = FALSE)
```

	Effect	r2_indirect	CI_Low	CI_High	Perc	CI_Low_P	CI_High_P
Med_Path1	[a1*b1]	0.00921	0.00408	0.01688	0.92%	0.41%	1.69%
Med_Path2	[a2*b2]	0.00173	0.00016	0.00471	0.17%	0.02%	0.47%
Serial_Path	[a1*d*b2]	0.01681	0.01161	0.02458	1.68%	1.16%	2.46%
Total Mediated Effect		0.07141	0.05273	0.09140	7.14%	5.27%	9.14%

```
cat("\n--> FUNCTION REPORT: DATASET 3 (TYPE 1 SUPPRESSION, N = 700)\n")
```

```
--> FUNCTION REPORT: DATASET 3 (TYPE 1 SUPPRESSION, N = 700)
```

```
print(res_dfs3, row.names = FALSE)
```

	Effect	r2_indirect	CI_Low	CI_High	Perc	CI_Low_P	CI_High_P
Med_Path1	[a1*b1]	0.09531	0.06972	0.12505	9.53%	6.97%	12.5%
Med_Path2	[a2*b2]	0.00988	0.00384	0.01946	0.99%	0.38%	1.95%
Serial_Path	[a1*d*b2]	0.02182	0.01415	0.03190	2.18%	1.41%	3.19%
Total Mediated Effect		0.00380	0.00039	0.01412	0.38%	0.04%	1.41%

```
cat("\n--> FUNCTION REPORT: DATASET 4 (TYPE 2 SUPPRESSION, N = 700)\n")
```

```
--> FUNCTION REPORT: DATASET 4 (TYPE 2 SUPPRESSION, N = 700)
```

```
print(res_dfs4, row.names = FALSE)
```

	Effect	r2_indirect	CI_Low	CI_High	Perc	CI_Low_P	CI_High_P
Med_Path1	[a1*b1]	0.06024	0.03891	0.08182	6.02%	3.89%	8.18%
Med_Path2	[a2*b2]	0.04816	0.03008	0.07081	4.82%	3.01%	7.08%
Serial_Path	[a1*d*b2]	0.01115	0.00685	0.01828	1.11%	0.68%	1.83%
Total Mediated Effect		0.00633	0.00037	0.01915	0.63%	0.04%	1.92%

```
cat("\n--> FUNCTION REPORT: DATASET 5 (TYPE 3 SUPPRESSION, N = 700)\n")
```

```
--> FUNCTION REPORT: DATASET 5 (TYPE 3 SUPPRESSION, N = 700)
```

```
print(res_dfs5, row.names = FALSE)
```

	Effect	r2_indirect	CI_Low	CI_High	Perc	CI_Low_P	CI_High_P
Med_Path1	[a1*b1]	0.03537	0.02116	0.05623	3.54%	2.12%	5.62%
Med_Path2	[a2*b2]	0.01372	0.00700	0.02371	1.37%	0.7%	2.37%
Serial_Path	[a1*d*b2]	0.02429	0.01586	0.03506	2.43%	1.59%	3.51%
Total Mediated Effect		0.02230	0.01080	0.04053	2.23%	1.08%	4.05%

```
cat("\n--> FUNCTION REPORT: DATASET 6 (TYPE 4 SUPPRESSION, N = 700)\n")
```

```
--> FUNCTION REPORT: DATASET 6 (TYPE 4 SUPPRESSION, N = 700)
```

```
print(res_dfs5, row.names = FALSE)
```

	Effect	r2_indirect	CI_Low	CI_High	Perc	CI_Low_P	CI_High_P
Med_Path1	[a1*b1]	0.03537	0.02116	0.05623	3.54%	2.12%	5.62%
Med_Path2	[a2*b2]	0.01372	0.00700	0.02371	1.37%	0.7%	2.37%
Serial_Path	[a1*d*b2]	0.02429	0.01586	0.03506	2.43%	1.59%	3.51%
Total Mediated Effect		0.02230	0.01080	0.04053	2.23%	1.08%	4.05%

```
#
=====
# METHODOLOGICAL CHECK: WRIGHT'S DECOMPOSITION FOR SERIAL MEDIATORS (15 BLOCKS)
#
=====
wright_proof_serial <- function(model_fit, dataset_name = "Dataset") {

  est <- parameterEstimates(model_fit)

  # Structural paths
  a1_est <- est$est[est$label == "a1"]
  b1_est <- est$est[est$label == "b1"]
```

```

a2_est <- est$est[est$label == "a2"]
b2_est <- est$est[est$label == "b2"]
d_est  <- est$est[est$label == "d"]
cp_est <- est$est[est$label == "cp"]

# Residual Variances
var_eM1 <- est$est[est$op == "~~" & est$lhs == "M1" & est$rhs == "M1"]
var_eM2 <- est$est[est$op == "~~" & est$lhs == "M2" & est$rhs == "M2"]
var_eY  <- est$est[est$op == "~~" & est$lhs == "Y"  & est$rhs == "Y"]

# Matrix Sample Variances
cov_mat <- lavInspect(model_fit, "cov.ov")
var_X   <- cov_mat["X", "X"]
var_Y   <- cov_mat["Y", "Y"]

# Compute the 15 Raw Components
# --- 1. Effects originating from X ---
B1_Raw <- (cp_est)^2 * var_X
B2_Raw <- (a1_est * b1_est)^2 * var_X
B3_Raw <- (a2_est * b2_est)^2 * var_X
B4_Raw <- (a1_est * d_est * b2_est)^2 * var_X
B5_Raw <- 2 * cp_est * (a1_est * b1_est) * var_X
B6_Raw <- 2 * cp_est * (a2_est * b2_est) * var_X
B7_Raw <- 2 * cp_est * (a1_est * d_est * b2_est) * var_X
B8_Raw <- 2 * (a1_est * b1_est) * (a2_est * b2_est) * var_X
B9_Raw <- 2 * (a1_est * b1_est) * (a1_est * d_est * b2_est) * var_X
B10_Raw <- 2 * (a2_est * b2_est) * (a1_est * d_est * b2_est) * var_X

# --- 2. Effects originating from eM1
B11_Raw <- (b1_est)^2 * var_eM1
B12_Raw <- (d_est * b2_est)^2 * var_eM1
B13_Raw <- 2 * b1_est * (d_est * b2_est) * var_eM1

# --- 3. Effects originating from eM2 and eY ---
B14_Raw <- (b2_est)^2 * var_eM2
B15_Raw <- var_eY

Total_Raw <- sum(B1_Raw, B2_Raw, B3_Raw, B4_Raw, B5_Raw, B6_Raw, B7_Raw,
                  B8_Raw, B9_Raw, B10_Raw, B11_Raw, B12_Raw, B13_Raw,
                  B14_Raw, B15_Raw)

# Proportions
P1 <- B1_Raw / var_Y; P2 <- B2_Raw / var_Y; P3 <- B3_Raw / var_Y

```

```

P4 <- B4_Raw / var_Y; P5 <- B5_Raw / var_Y; P6 <- B6_Raw / var_Y
P7 <- B7_Raw / var_Y; P8 <- B8_Raw / var_Y; P9 <- B9_Raw / var_Y
P10 <- B10_Raw / var_Y; P11 <- B11_Raw / var_Y; P12 <- B12_Raw / var_Y
P13 <- B13_Raw / var_Y; P14 <- B14_Raw / var_Y; P15 <- B15_Raw / var_Y

Total_Prop <- sum(P1, P2, P3, P4, P5, P6, P7, P8, P9, P10, P11, P12, P13,
P14, P15)

# --- PRINTING THE COMPREHENSIVE MATHEMATICAL CHECK REPORT ---
cat("\n=====\\n")
cat(" WRIGHT DECOMPOSITION PROOF (SERIAL):", dataset_name, "\\n")
cat("=====\\n")
cat(sprintf(" Observed Variance of Y (Denominator) : %10.5f\\n", var_Y))
cat(sprintf(" Implied Variance of Y (Reconstructed): %10.5f\\n", Total_Raw))
cat(sprintf(" Discrepancy (Reconstruction Error) : %10.5f\\n", var_Y -
Total_Raw))
cat("-----\\n")
cat(sprintf(" %-38s | %-10s | %-10s | %-10s\\n", "COMPONENT", "RAW VAR",
"PROPORTION", "PERCENT"))
cat("-----\\n")
cat(sprintf(" 1. Pure Direct Effect (cp^2) | %10.5f | %10.5f | %8.2f
%%\\n", B1_Raw, P1, P1*100))
cat(sprintf(" 2. Pure Indirect M1 ((a1*b1)^2) | %10.5f | %10.5f | %8.2f
%%\\n", B2_Raw, P2, P2*100))
cat(sprintf(" 3. Pure Indirect M2 ((a2*b2)^2) | %10.5f | %10.5f | %8.2f
%%\\n", B3_Raw, P3, P3*100))
cat(sprintf(" 4. Pure Serial ((a1*d*b2)^2) | %10.5f | %10.5f | %8.2f
%%\\n", B4_Raw, P4, P4*100))
cat(sprintf(" 5. Wright Joint Cov (2*cp*a1*b1) | %10.5f | %10.5f | %8.2f
%%\\n", B5_Raw, P5, P5*100))
cat(sprintf(" 6. Wright Joint Cov (2*cp*a2*b2) | %10.5f | %10.5f | %8.2f
%%\\n", B6_Raw, P6, P6*100))
cat(sprintf(" 7. Wright Joint Cov (2*cp*a1*d*b2) | %10.5f | %10.5f | %8.2f
%%\\n", B7_Raw, P7, P7*100))
cat(sprintf(" 8. Wright Joint Cov (2*a1*b1*a2*b2) | %10.5f | %10.5f | %8.2f
%%\\n", B8_Raw, P8, P8*100))
cat(sprintf(" 9. Wright Joint Cov (2*a1*b1*a1*d*b2) | %10.5f | %10.5f | %8.2f
%%\\n", B9_Raw, P9, P9*100))
cat(sprintf(" 10. Wright Joint Cov (2*a2*b2*a1*d*b2) | %10.5f | %10.5f | %8.2f
%%\\n", B10_Raw, P10, P10*100))
cat(sprintf(" 11. Indep. M1 Error via b1 (b1^2*eM1) | %10.5f | %10.5f | %8.2f

```

```

%%\n", B11_Raw, P11, P11*100))
  cat(sprintf(" 12. Indep. M1 Error via M2 ((d*b2)^2*e)| %10.5f | %10.5f | %8.2f
%%\n", B12_Raw, P12, P12*100))
  cat(sprintf(" 13. M1 Error Joint Cov (2*b1*d*b2*eM1) | %10.5f | %10.5f | %8.2f
%%\n", B13_Raw, P13, P13*100))
  cat(sprintf(" 14. Indep. M2 Error (b2^2*eM2) | %10.5f | %10.5f | %8.2f
%%\n", B14_Raw, P14, P14*100))
  cat(sprintf(" 15. Y Residual/Error Var (eY) | %10.5f | %10.5f | %8.2f
%%\n", B15_Raw, P15, P15*100))
  cat("-----
\n")
  cat(sprintf(" %-38s | %10.5f | %10.5f | %8.2f %%\n", "TOTAL CHECK", Total_Raw,
Total_Prop, Total_Prop*100))
  cat("=====\\n")
}

#
=====
# EXECUTE PROOF ON THE FITTED MODELS
#
=====
# Assuming models are already estimated in lavaan using raw data:
wright_proof_serial(fits1, "Dataset 1 (N=600, Standard)")

```

=====

WRIGHT DECOMPOSITION PROOF (SERIAL): Dataset 1 (N=600, Standard)

=====

Observed Variance of Y (Denominator) : 2.35446
Implied Variance of Y (Reconstructed): 2.35446
Discrepancy (Reconstruction Error) : -0.00000

COMPONENT	RAW VAR	PROPORTION	PERCENT
1. Pure Direct Effect (cp^2)	0.07538	0.03201	3.20 %
2. Pure Indirect M1 ((a1*b1)^2)	0.05181	0.02200	2.20 %
3. Pure Indirect M2 ((a2*b2)^2)	0.03707	0.01575	1.57 %
4. Pure Serial ((a1*d*b2)^2)	0.01296	0.00551	0.55 %
5. Wright Joint Cov (2*cp*a1*b1)	0.12498	0.05308	5.31 %
6. Wright Joint Cov (2*cp*a2*b2)	0.10573	0.04490	4.49 %
7. Wright Joint Cov (2*cp*a1*d*b2)	0.06252	0.02655	2.66 %
8. Wright Joint Cov (2*a1*b1*a2*b2)	0.08765	0.03723	3.72 %

9. Wright Joint Cov ($2*a1*b1*a1*d*b2$)		0.05183		0.02201		2.20 %
10. Wright Joint Cov ($2*a2*b2*a1*d*b2$)		0.04384		0.01862		1.86 %
11. Indep. M1 Error via b1 ($b1^2*eM1$)		0.19042		0.08088		8.09 %
12. Indep. M1 Error via M2 ($(d*b2)^2*e$)		0.04764		0.02024		2.02 %
13. M1 Error Joint Cov ($2*b1*d*b2*eM1$)		0.19050		0.08091		8.09 %
14. Indep. M2 Error ($b2^2*eM2$)		0.25524		0.10841		10.84 %
15. Y Residual/Error Var (eY)		1.01688		0.43190		43.19 %

TOTAL CHECK		2.35446		1.00000		100.00 %
=====						

```
wright_proof_serial(fits2, "Dataset 2 (N=700, Standard)")
```

=====				
WRIGHT DECOMPOSITION PROOF (SERIAL): Dataset 2 (N=700, Standard)				
=====				
Observed Variance of Y (Denominator) :		2.26020		
Implied Variance of Y (Reconstructed):		2.26020		
Discrepancy (Reconstruction Error) :		0.00000		

COMPONENT		RAW VAR	PROPORTION	PERCENT

1. Pure Direct Effect (cp^2)		0.19174	0.08483	8.48 %
2. Pure Indirect M1 ($(a1*b1)^2$)		0.02081	0.00921	0.92 %
3. Pure Indirect M2 ($(a2*b2)^2$)		0.00391	0.00173	0.17 %
4. Pure Serial ($(a1*d*b2)^2$)		0.03800	0.01681	1.68 %
5. Wright Joint Cov ($2*cp*a1*b1$)		0.12635	0.05590	5.59 %
6. Wright Joint Cov ($2*cp*a2*b2$)		0.05477	0.02423	2.42 %
7. Wright Joint Cov ($2*cp*a1*d*b2$)		0.17071	0.07553	7.55 %
8. Wright Joint Cov ($2*a1*b1*a2*b2$)		0.01804	0.00798	0.80 %
9. Wright Joint Cov ($2*a1*b1*a1*d*b2$)		0.05624	0.02488	2.49 %
10. Wright Joint Cov ($2*a2*b2*a1*d*b2$)		0.02438	0.01079	1.08 %
11. Indep. M1 Error via b1 ($b1^2*eM1$)		0.05677	0.02512	2.51 %
12. Indep. M1 Error via M2 ($(d*b2)^2*e$)		0.10363	0.04585	4.59 %
13. M1 Error Joint Cov ($2*b1*d*b2*eM1$)		0.15340	0.06787	6.79 %
14. Indep. M2 Error ($b2^2*eM2$)		0.26021	0.11512	11.51 %
15. Y Residual/Error Var (eY)		0.98125	0.43414	43.41 %

TOTAL CHECK		2.26020	1.00000	100.00 %
=====				

```
wright_proof_serial(fits3, "Dataset 3 (N=700, Type 1 Suppression)")
```

=====			
WRIGHT DECOMPOSITION PROOF (SERIAL): Dataset 3 (N=700, Type 1 Suppression)			
=====			
Observed Variance of Y (Denominator) :	1.66866		
Implied Variance of Y (Reconstructed):	1.66866		
Discrepancy (Reconstruction Error) :	0.00000		

COMPONENT	RAW VAR	PROPORTION	PERCENT

1. Pure Direct Effect (cp^2)	0.42937	0.25731	25.73 %
2. Pure Indirect M1 ($(a1*b1)^2$)	0.15904	0.09531	9.53 %
3. Pure Indirect M2 ($(a2*b2)^2$)	0.01648	0.00988	0.99 %
4. Pure Serial ($(a1*d*b2)^2$)	0.03641	0.02182	2.18 %
5. Wright Joint Cov ($2*cp*a1*b1$)	-0.52264	-0.31321	-31.32 %
6. Wright Joint Cov ($2*cp*a2*b2$)	0.16824	0.10082	10.08 %
7. Wright Joint Cov ($2*cp*a1*d*b2$)	0.25007	0.14986	14.99 %
8. Wright Joint Cov ($2*a1*b1*a2*b2$)	-0.10239	-0.06136	-6.14 %
9. Wright Joint Cov ($2*a1*b1*a1*d*b2$)	-0.15220	-0.09121	-9.12 %
10. Wright Joint Cov ($2*a2*b2*a1*d*b2$)	0.04899	0.02936	2.94 %
11. Indep. M1 Error via b1 ($b1^2*eM1$)	0.35105	0.21038	21.04 %
12. Indep. M1 Error via M2 ($(d*b2)^2*e$)	0.08037	0.04816	4.82 %
13. M1 Error Joint Cov ($2*b1*d*b2*eM1$)	-0.33594	-0.20132	-20.13 %
14. Indep. M2 Error ($b2^2*eM2$)	0.34836	0.20876	20.88 %
15. Y Residual/Error Var (eY)	0.89344	0.53543	53.54 %

TOTAL CHECK	1.66866	1.00000	100.00 %
=====			

```
wright_proof_serial(fits4, "Dataset 4 (N=700, Type 2 Suppression)")
```

=====			
WRIGHT DECOMPOSITION PROOF (SERIAL): Dataset 4 (N=700, Type 2 Suppression)			
=====			
Observed Variance of Y (Denominator) :	1.47811		
Implied Variance of Y (Reconstructed):	1.47811		
Discrepancy (Reconstruction Error) :	0.00000		

COMPONENT	RAW VAR	PROPORTION	PERCENT
1. Pure Direct Effect (cp^2)	0.05758	0.03895	3.90 %
2. Pure Indirect M1 ($(a1*b1)^2$)	0.08904	0.06024	6.02 %
3. Pure Indirect M2 ($(a2*b2)^2$)	0.07118	0.04816	4.82 %
4. Pure Serial ($(a1*d*b2)^2$)	0.01647	0.01115	1.11 %
5. Wright Joint Cov ($2*cp*a1*b1$)	0.14321	0.09689	9.69 %
6. Wright Joint Cov ($2*cp*a2*b2$)	-0.12804	-0.08662	-8.66 %
7. Wright Joint Cov ($2*cp*a1*d*b2$)	-0.06160	-0.04167	-4.17 %
8. Wright Joint Cov ($2*a1*b1*a2*b2$)	-0.15923	-0.10772	-10.77 %
9. Wright Joint Cov ($2*a1*b1*a1*d*b2$)	-0.07660	-0.05182	-5.18 %
10. Wright Joint Cov ($2*a2*b2*a1*d*b2$)	0.06849	0.04634	4.63 %
11. Indep. M1 Error via b1 ($b1^2*eM1$)	0.22700	0.15358	15.36 %
12. Indep. M1 Error via M2 ($(d*b2)^2*e$)	0.04200	0.02841	2.84 %
13. M1 Error Joint Cov ($2*b1*d*b2*eM1$)	-0.19528	-0.13212	-13.21 %
14. Indep. M2 Error ($b2^2*eM2$)	0.33439	0.22623	22.62 %
15. Y Residual/Error Var (eY)	1.04949	0.71002	71.00 %
TOTAL CHECK	1.47811	1.00000	100.00 %

```
wright_proof_serial(fits5, "Dataset 3 (N=700, Type 3 Suppression)")
```

WRIGHT DECOMPOSITION PROOF (SERIAL): Dataset 3 (N=700, Type 3 Suppression)			
Observed Variance of Y (Denominator) : 1.74804			
Implied Variance of Y (Reconstructed): 1.74804			
Discrepancy (Reconstruction Error) : 0.00000			
COMPONENT	RAW VAR	PROPORTION	PERCENT
1. Pure Direct Effect (cp^2)	0.21091	0.12066	12.07 %
2. Pure Indirect M1 ($(a1*b1)^2$)	0.06182	0.03537	3.54 %
3. Pure Indirect M2 ($(a2*b2)^2$)	0.02398	0.01372	1.37 %
4. Pure Serial ($(a1*d*b2)^2$)	0.04246	0.02429	2.43 %
5. Wright Joint Cov ($2*cp*a1*b1$)	0.22838	0.13065	13.06 %
6. Wright Joint Cov ($2*cp*a2*b2$)	0.14223	0.08136	8.14 %
7. Wright Joint Cov ($2*cp*a1*d*b2$)	-0.18927	-0.10828	-10.83 %
8. Wright Joint Cov ($2*a1*b1*a2*b2$)	0.07700	0.04405	4.41 %
9. Wright Joint Cov ($2*a1*b1*a1*d*b2$)	-0.10247	-0.05862	-5.86 %

10. Wright Joint Cov ($2*a2*b2*a1*d*b2$)	-0.06382	-0.03651	-3.65 %
11. Indep. M1 Error via b1 ($b1^2*eM1$)	0.18714	0.10706	10.71 %
12. Indep. M1 Error via M2 ($(d*b2)^2*e$)	0.12854	0.07353	7.35 %
13. M1 Error Joint Cov ($2*b1*d*b2*eM1$)	-0.31019	-0.17745	-17.75 %
14. Indep. M2 Error ($b2^2*eM2$)	0.25628	0.14661	14.66 %
15. Y Residual/Error Var (eY)	1.05505	0.60356	60.36 %

TOTAL CHECK	1.74804	1.00000	100.00 %
=====			

wright_proof_serial(fits6, "Dataset 4 (N=700, Type 4 Suppression)")

=====			
WRIGHT DECOMPOSITION PROOF (SERIAL): Dataset 4 (N=700, Type 4 Suppression)			
=====			
Observed Variance of Y (Denominator) :	1.94579		
Implied Variance of Y (Reconstructed):	1.94579		
Discrepancy (Reconstruction Error) :	0.00000		

COMPONENT	RAW VAR	PROPORTION	PERCENT

1. Pure Direct Effect (cp^2)	0.24465	0.12573	12.57 %
2. Pure Indirect M1 ($(a1*b1)^2$)	0.05888	0.03026	3.03 %
3. Pure Indirect M2 ($(a2*b2)^2$)	0.02398	0.01232	1.23 %
4. Pure Serial ($(a1*d*b2)^2$)	0.04431	0.02277	2.28 %
5. Wright Joint Cov ($2*cp*a1*b1$)	-0.24005	-0.12337	-12.34 %
6. Wright Joint Cov ($2*cp*a2*b2$)	-0.15318	-0.07872	-7.87 %
7. Wright Joint Cov ($2*cp*a1*d*b2$)	-0.20823	-0.10702	-10.70 %
8. Wright Joint Cov ($2*a1*b1*a2*b2$)	0.07515	0.03862	3.86 %
9. Wright Joint Cov ($2*a1*b1*a1*d*b2$)	0.10216	0.05250	5.25 %
10. Wright Joint Cov ($2*a2*b2*a1*d*b2$)	0.06519	0.03350	3.35 %
11. Indep. M1 Error via b1 ($b1^2*eM1$)	0.17824	0.09160	9.16 %
12. Indep. M1 Error via M2 ($(d*b2)^2*e$)	0.13412	0.06893	6.89 %
13. M1 Error Joint Cov ($2*b1*d*b2*eM1$)	0.30923	0.15892	15.89 %
14. Indep. M2 Error ($b2^2*eM2$)	0.25628	0.13171	13.17 %
15. Y Residual/Error Var (eY)	1.05505	0.54222	54.22 %

TOTAL CHECK	1.94579	1.00000	100.00 %
=====			

#####

5. A-Priori Power Analysis and Sensitivity Analysis based on R^2_{indirect}

5.1 Theoretical Framework: Joint Significance Power

Statistical power in mediation analysis has been evaluated using individual path coefficients rather than a consolidated effect size metric. This framework operationalizes power analysis directly through the **Joint Significance Criterion**. Under this approach, an indirect effect is deemed statistically significant if and only if every single constituent path coefficient within the structural chain ($X \rightarrow M \rightarrow Y$) is significantly different from zero at a designated alpha level (α).

Assuming the sampling distributions of the path coefficients are asymptotically independent, the joint power of the indirect pathway ($\text{Power}_{\text{joint}}$) is defined as the product of the individual statistical powers of its k constituent paths:

$$\text{Power}_{\text{joint}} = \prod_{i=1}^k \text{Power}_i$$

Because $\text{Power}_i \leq 1$, the joint power decreases as the length of the mediation chain (k_paths) increases. This mathematical reality requires sample size planning to account for the compounding erosion of power across multi-stage structures.

5.2 Mathematical Boundary Profiles: Balanced vs. Bottleneck Scenarios

A known mathematical property of any product-based effect size—including the unstandardized product $a \cdot b$, the standardized product $\beta_a \cdot \beta_b$, and the variance-accounted-for metric R^2_{indirect} —is that an infinite number of path combinations can yield the exact same global effect size. For instance, an $R^2_{\text{indirect}} = 0.01$ can result from two moderate paths or from one extremely large path paired with a near-zero path.

To resolve this indeterminacy without relying on arbitrary parameter assumptions, this framework implements a boundary-profile method that defines two structural extremes:

- **The Balanced Scenario (Best Case):** This profile assumes that the underlying structural mechanism is perfectly symmetrical. The total variance explained by the indirect effect is distributed evenly across all k paths. For a target indirect coefficient $K = \sqrt{R^2_{\text{indirect}}}$, the standardized coefficient for each path is calculated as:

$$\beta_{\text{balanced}} = K^{\frac{1}{k}}$$

Because statistical power is a non-linear function maximized when the minimum effect size in a chain is as large as possible, the balanced scenario yields the highest possible joint power for a given R^2_{indirect} . It provides the lower bound for the required sample size (N).

- **The Bottleneck Scenario (Worst Case):** This profile assumes extreme structural asymmetry. One path acts as a severe statistical restriction (the “bottleneck”), while the remaining $k - 1$ paths are maximized up to a user-defined empirical ceiling ($\beta_{\max}$, or `max_beta`). The bottleneck path coefficient ($\beta_{\min}$) is formulated as:

$$\beta_{\min} = \frac{K}{(\beta_{\max})^{k-1}}$$

This scenario represents the most conservative estimation layout. It identifies the minimum sample size required to detect the effect even if the mediation chain relies on a single highly fragile pathway.

5.3 Error Variance Adjustments and Multi-Mediator Penalties

The power engine does not treat the outcome variable’s residual variance as a static parameter. Following Wright’s path tracing rules, the residual variance of the outcome ($\sigma_{e_Y}^2$) is dynamically computed by subtracting all explained components from the total unit variance (1):

$$\sigma_{e_Y}^2 = 1 - (c'^2 + K^2 + 2c'K) - R_{\text{parallel}}^2 - R_{\text{covariates}}^2$$

Where c' is the direct effect, K is the indirect path coefficient ($\sqrt{R_{\text{indirect}}^2}$), $2c'K$ is the joint covariance block, R_{parallel}^2 is the variance explained by alternative parallel paths, and $R_{\text{covariates}}^2$ represents external control variables.

Furthermore, when evaluating a target mediator embedded within a parallel mediation design, the framework applies an analytic Variance Inflation Factor (VIF) penalty to account for collinearity among mediators:

$$\text{VIF} = \frac{1}{1 - R_{\text{m_overlap}}^2}$$

This penalty inflates the standard errors of the path coefficients, capturing the loss of statistical precision that occurs when parallel mediators share overlapping variance ($R_{\text{m_overlap}}^2$).

5.4 The Empirical Guardrail (`max_beta`)

In structural equation modeling, standardized path coefficients represent partial correlations or standardized regression weights. Mathematically, they are generally bounded by an absolute value of 1.0. Furthermore, in empirical behavioral and social sciences, it is extremely rare for a single structural path to exceed 0.80 without indicating severe theoretical redundancy or multi-collinearity issues.

When the algorithm computes the Bottleneck Scenario, it forces one path to be as large as possible to evaluate the worst-case structural asymmetry. Without a cap, the mathematical engine could theoretically maximize this path up to 0.99 to satisfy the required R_{indirect}^2 , resulting in an empirically absurd profile.

To prevent this, the framework introduces the `max_beta` parameter as an empirical guardrail (default to 0.80). It strictly restricts the maximum possible magnitude for any single constituent path

in the mediation chain. If achieving a specific effect size requires a structural path that exceeds this limit, the algorithm halts and correctly flags the scenario as "Unreachable", acknowledging that such a structure violates empirical plausibility.

6. Power Analysis for Single, Parallel, and Serial Mediation (wright_power)

6.1 Algorithmic Mechanics

The `wright_power` function computes the required sample size (N) to detect a target R^2_{indirect} by executing a deterministic grid-search optimization across a range of $N \in [10, 5000]$.

For each iteration of N , the function executes the following mathematical sequence:

1. **Standard Error Calculation:** It calculates the standard error for any given path coefficient (β) based on the current sample size N and the dynamic residual variance:

$$SE_{\beta} = \sqrt{\frac{\sigma_{eY}^2}{N \cdot (1 - \beta^2)}} \cdot \sqrt{VIF}$$

2. **Individual Path Power:** It computes the statistical power for each distinct structural pathway using the standard normal cumulative distribution function (Φ):

$$\text{Power}_{\beta} = \Phi\left(\frac{\beta}{SE_{\beta}} - z_{1-\frac{\alpha}{2}}\right)$$

3. **Joint Power Aggregation:** It aggregates the individual path powers into the final joint power metric using a multiplicative probability function:
 - **For the Balanced Scenario:** $\text{Power}_{\text{joint}} = (\text{Power}_{\text{balanced}})^k$
 - **For the Bottleneck Scenario:** $\text{Power}_{\text{joint}} = \text{Power}_{\text{min}} \cdot (\text{Power}_{\text{max}})^{k-1}$
4. **Identifying Required Sample Size:** The sequence iterates, increasing N , until it identifies the exact sample sizes where the calculated joint power successfully crosses the user-specified threshold (`power_target`).

6.2 Structural Adaptation (Single, Parallel, and Serial Models)

The function adapts to different structural architectures via specific execution arguments:

- **Single Mediation:** Evaluated by setting `k_paths = 2`, `r2_parallel = 0`, and `r2_m_overlap = 0`.
- **Parallel Mediation:** Incorporates the variance drainage of concurrent pathways via `r2_parallel` and handles the structural collinearity of competing mediators by setting `r2_m_overlap > 0` (triggering the VIF penalty).
- **Serial Mediation:** Evaluated by scaling `k_paths` to 3 or more structural steps. The mathematical engine automatically projects the compounding power degradation across the multi-stage chain.

Additionally, the algorithm also accounts for the directionality of the structural paths via the `type` parameter. Because the decomposition of variance includes a joint covariance block ($2c'K$), the signs of the effects dictate the final residual error of the outcome (σ_{eY}^2):

- **Consistent Models (`type = "consistent"`):** The direct effect (c') and the indirect effect have the same sign. They cooperate to explain variance in Y , mathematically shrinking the residual error. This deflation of error variance tightens the standard errors of the paths, thereby increasing statistical power and lowering the required sample size.
- **Suppression Models (`type = "suppression"`):** The direct and indirect effects have opposite signs. Their mathematical combination cancels out explained variance, artificially inflating the residual error (σ_{eY}^2). This inflation bloats the standard errors, drastically reducing power and requiring a significantly larger sample size to detect the same R_{indirect}^2 .
- **Explore Mode (`type = "explore"`):** The default setting forces the engine to run the optimization twice—once under a consistent regime and once under a suppression regime. This provides the researcher with a complete bounding profile, defining both the best-case and worst-case sample size requirements based on path directionality.

```
#' Wright Mediation Power Analysis (A-Priori)
#
# Computes the required Sample Size (N) to detect a specific indirect R2,
# returning the Best (Balanced) and Worst (Bottleneck) scenarios.
#
# @param r2_indirect Target variance of Y explained by the indirect effect.
# @param type "explore" (default), "consistent", or "suppression".
# @param k_paths Number of multiplicative paths in the indirect block (default
# = 2).
# @param c_prime Expected direct effect coefficient (default = 0.20).
# @param r2_m_overlap:
# Variance of the target mediator explained by other parallel mediators (VIF
# penalty).
# Tip: For 2 mediators with a correlation of 0.30, enter the squared value
# (0.30^2 = 0.09).
# Tip: For 3+ mediators, enter the multiple R2 of the target regressed on
# the others.
# @param r2_parallel Variance of Y explained by other parallel mediators.
# @param r2_covariates Variance of Y explained by external control variables.
# @param power_target Desired joint statistical power threshold (default =
# 0.80).
# @param alpha Significance level (default = 0.05).
# @param max_beta Maximum allowed single standardized coefficient (guardrail,
# default = 0.80).

wright_power <- function(r2_indirect, type = "explore", k_paths = 2, c_prime
= 0.20,
```

```

    r2_m_overlap = 0, r2_parallel = 0, r2_covariates = 0,
    power_target = 0.80, alpha = 0.05, max_beta = 0.80) {

# 1. INPUT VALIDATION
  if(r2_indirect <= 0 | r2_indirect >= 1) stop("r2_indirect must be between 0
and 1.")
  if(r2_covariates < 0 | r2_covariates >= 0.90) stop("r2_covariates must be
between 0 and 0.90.")
  if(r2_parallel < 0 | r2_parallel >= 0.90) stop("r2_parallel must be between 0
and 0.90.")
  if(r2_m_overlap < 0 | r2_m_overlap >= 1) stop("r2_m_overlap must be strictly
between 0 and 1.")
  if(k_paths < 2) stop("k_paths must be at least 2.")

# 2. PARALLEL MEDIATION PENALTY (Analytic VIF)
  vif_penalty <- 1 / (1 - r2_m_overlap)

# 3. CORE POWER ENGINE
  get_n <- function(cp) {
    K <- sqrt(r2_indirect)
    min_beta <- K / (max_beta^(k_paths - 1))

    # Skip if constraints violate the geometric guardrail
    if(min_beta > max_beta) return(c(NA, NA))

    # Compute residual variance of Y (incorporating covariates and parallel
mediators)
    var_error_y <- 1 - (cp^2 + K^2 + 2 * cp * K) - r2_covariates - r2_parallel
    if(var_error_y <= 0) var_error_y <- 0.01

    # Grid search for N
    n_grid <- seq(10, 5000, by = 1)
    n_balanced <- NA
    n_bottleneck <- NA

    for(n in n_grid) {
      # --- BALANCED SCENARIO ---
      beta_bal <- K^(1/k_paths)
      se_bal <- sqrt(var_error_y / (n * (1 - beta_bal^2))) * sqrt(vif_penalty)
      pow_bal <- pnorm((beta_bal / se_bal) - qnorm(1 - alpha/2))
      joint_pow_bal <- pow_bal^k_paths

      # --- BOTTLENECK SCENARIO ---

```

```

se_min <- sqrt(var_error_y / (n * (1 - min_beta^2))) * sqrt(vif_penalty)
se_max <- sqrt(var_error_y / (n * (1 - max_beta^2))) * sqrt(vif_penalty)
pow_min <- pnorm((min_beta / se_min) - qnorm(1 - alpha/2))
pow_max <- pnorm((max_beta / se_max) - qnorm(1 - alpha/2))
joint_pow_bot <- pow_min * (pow_max^(k_paths - 1))

# Capture threshold crossing points
if(is.na(n_balanced) && joint_pow_bal >= power_target) n_balanced <- n
if(is.na(n_bottleneck) && joint_pow_bot >= power_target) n_bottleneck <- n

# Break optimization if both profiles are populated
if(!is.na(n_balanced) && !is.na(n_bottleneck)) break
}
return(c(n_balanced, n_bottleneck))
}

# 4. OUTPUT GENERATION
cat("\n===== \n")
cat("          WRIGHT MEDIATION A PRIORI POWER ANALYSIS          \n")
cat("===== \n")
cat(sprintf("Target Indirect R2:      %.4f \n", r2_indirect))
cat(sprintf("Structural Paths:          %d \n", k_paths))
cat(sprintf("Direct Effect (c'):        %.2f (Base Magnitude)\n", abs(c_prime)))
cat(sprintf("Other Mediators (R2):      %.2f (Variance in Y)\n", r2_parallel))
cat(sprintf("Target M Overlap (R2):    %.2f (Collinearity VIF = %.2f)\n",
r2_m_overlap, vif_penalty))
cat(sprintf("Covariates (R2):          %.2f \n", r2_covariates))
cat(sprintf("Target Joint Power:      %.2f \n", power_target))
cat("----- \n")
cat(sprintf("%-20s %-18s %-12s\n", "SCENARIO", "MODEL TYPE", "REQUIRED N"))
cat("----- \n")

scenarios <- if(type == "explore") c("consistent", "suppression") else type

for(t in scenarios) {
  cp_val <- if(t == "consistent") abs(c_prime) else -abs(c_prime)
  t_label <- if(t == "consistent") "Consistent" else "Suppression"

  results <- get_n(cp_val)

  n_bal <- if(!is.na(results[1])) sprintf("%d", results[1]) else "Unreachable"
  n_bot <- if(!is.na(results[2])) sprintf("%d", results[2]) else "Unreachable"

```



```

cat(sprintf("%-20s %-18s %-12s\n", "Balanced", t_label, n_bal))
cat(sprintf("%-20s %-18s %-12s\n", "Bottleneck", t_label, n_bot))
if(t != scenarios[length(scenarios)]) cat("- - - - -\n")
- - - - -\n")
}
cat("=====\n")
cat("Method: Joint Significance Power.\n\n")
}

#####
###Function usage example!

#Single mediator model
wright_power(r2_indirect = 0.0081,
             type = "explore",
             k_paths = 2,
             c_prime = 0.30,
             r2_m_overlap = 0,
             r2_parallel = 0,
             r2_covariates = 0,
             power_target = 0.80,
             alpha = 0.05,
             max_beta = 0.80)

```

```

=====
                WRIGHT MEDIATION A PRIORI POWER ANALYSIS
=====
Target Indirect R2:    0.0081
Structural Paths:      2
Direct Effect (c'):    0.30 (Base Magnitude)
Other Mediators (R2):  0.00 (Variance in Y)
Target M Overlap (R2): 0.00 (Collinearity VIF = 1.00)
Covariates (R2):       0.00
Target Joint Power:    0.80
-----
SCENARIO           MODEL TYPE           REQUIRED N
-----
Balanced            Consistent            107
Bottleneck          Consistent            533
- - - - -

```

Balanced	Suppression	121
Bottleneck	Suppression	601

Method: Joint Significance Power.

```
#Two parallel mediators model
wright_power(r2_indirect = 0.0081,
             type = "explore",
             k_paths = 2,
             c_prime = 0.30,
             r2_m_overlap = 0.09,
             r2_parallel = 0.09,
             r2_covariates = 0,
             power_target = 0.80,
             alpha = 0.05,
             max_beta = 0.80)
```

WRIGHT MEDIATION A PRIORI POWER ANALYSIS

Target Indirect R2: 0.0081
 Structural Paths: 2
 Direct Effect (c'): 0.30 (Base Magnitude)
 Other Mediators (R2): 0.09 (Variance in Y)
 Target M Overlap (R2): 0.09 (Collinearity VIF = 1.10)
 Covariates (R2): 0.00
 Target Joint Power: 0.80

SCENARIO	MODEL TYPE	REQUIRED N
Balanced	Consistent	105
Bottleneck	Consistent	524
Balanced	Suppression	120
Bottleneck	Suppression	598

Method: Joint Significance Power.

```
#Two serial mediators model
wright_power(r2_indirect = 0.000729,
```

```

type = "explore",
k_paths = 3,
r2_m_overlap = 0,
r2_parallel = 0,
r2_covariates = 0,
power_target = 0.80,
alpha = 0.05,
max_beta = 0.80)

```

```

=====
                    WRIGHT MEDIATION A PRIORI POWER ANALYSIS
=====
Target Indirect R2:    0.0007
Structural Paths:      3
Direct Effect (c'):    0.20 (Base Magnitude)
Other Mediators (R2):  0.00 (Variance in Y)
Target M Overlap (R2): 0.00 (Collinearity VIF = 1.00)
Covariates (R2):       0.00
Target Joint Power:    0.80
-----
SCENARIO              MODEL TYPE              REQUIRED N
-----
Balanced              Consistent              136
Bottleneck            Consistent              4191
-----
Balanced              Suppression              139
Bottleneck            Suppression              4286
=====
Method: Joint Significance Power.

```

7. Sensitivity Analysis for the MDES (`wright_sensitivity`)

7.1 Algorithmic Mechanics

When the sample size (N) is predetermined by resource constraints, retrospective design limitations, or existing datasets, the analytical objective shifts from finding N to defining the **Minimum Detectable Effect Size (MDES)**. The `wright_sensitivity` function executes this by reverse-engineering the power calculation through a step-by-step search procedure:

1. **The Grid Search:** The function initiates a loop, starting from a near-zero indirect effect ($R^2_{\text{indirect}} = 0.0001$) and incrementally increasing it step-by-step up to ($R^2_{\text{indirect}} = 0.8000$).

2. **Parameter Extraction:** For each discrete variance value in the grid, the algorithm calculates exactly what the underlying path coefficients (β) would need to be under both the Balanced and Bottleneck structural profiles.
3. **Error Variance Computation:** Using Wright's decomposition, it calculates the dynamic residual variance of the outcome (σ_{eY}^2) associated with that specific structural configuration, accounting for any suppression or consistency effects.
4. **Power Evaluation:** Using the fixed sample size N provided by the user, it calculates the joint statistical power that would be achieved for that specific R_{indirect}^2 .
5. **Identifying the MDES:** The sequence iterates until the exact moment the calculated joint power crosses the user's specified threshold (e.g., 0.80). The loop then stops, and that specific R_{indirect}^2 value is returned as the MDES. It represents the absolute smallest variance proportion the study is statistically equipped to detect reliably.

7.2 Structural Interpretation of Outputs

The output of `wright_sensitivity` provides a clear assessment of a study's empirical threshold:

Scenario	Model Type	Required (MDES)	R_{indirect}^2	Interpretation
Balanced	Consistent / Suppression	<i>Value</i>		The absolute lowest variance proportion the study can detect under optimal, symmetrical path conditions.
Bottleneck	Consistent / Suppression	<i>Value</i>		The minimum variance proportion required if the model contains a highly asymmetric, weak pathway.

If a calculated MDES scenario returns "`Unreachable`", it signifies that the structural parameters, background covariates, or collinearity penalties drain so much variance from the system that it is mathematically impossible to achieve the target power at that sample size, regardless of the size of the indirect effect.

```
#' Wright Mediation Sensitivity Analysis
#
# Computes the Minimum Detectable Effect Size (MDES) in terms of indirect R2
# given a fixed sample size, returning the Best (Balanced) and Worst (Bottleneck)
# scenarios.
#
```

```

#' @param n Fixed sample size available for the study.
#' @param type "explore" (default), "consistent", or "suppression".
#' @param k_paths Number of multiplicative paths in the indirect block (default
= 2).
#' @param c_prime Expected direct effect coefficient (default = 0.20).
#' @param r2_m_overlap Variance of the target mediator explained by other parallel
mediators (VIF penalty).
#'      Tip: For 2 mediators with a correlation of 0.30, enter the squared
value ( $0.30^2 = 0.09$ ).
#'      Tip: For 3+ mediators, enter the multiple R2 of the target regressed
on the others.
#' @param r2_parallel Variance of Y explained by other parallel mediators.
#' @param r2_covariates Variance of Y explained by external control variables.
#' @param power_target Desired joint statistical power threshold (default =
0.80).
#' @param alpha Significance level (default = 0.05).
#' @param max_beta Maximum allowed single standardized coefficient (guardrail,
default = 0.80).

wright_sensitivity <- function(n, type = "explore", k_paths = 2, c_prime = 0.20,
                             r2_m_overlap = 0, r2_parallel = 0, r2_covariates
= 0,
                             power_target = 0.80, alpha = 0.05, max_beta =
0.80) {

  # 1. INPUT VALIDATION
  if(n < 10) stop("Sample size 'n' must be at least 10.")
  if(r2_covariates < 0 | r2_covariates >= 0.90) stop("r2_covariates must be
between 0 and 0.90.")
  if(r2_parallel < 0 | r2_parallel >= 0.90) stop("r2_parallel must be between 0
and 0.90.")
  if(r2_m_overlap < 0 | r2_m_overlap >= 1) stop("r2_m_overlap must be strictly
between 0 and 1.")
  if(k_paths < 2) stop("k_paths must be at least 2.")

  # 2. PARALLEL MEDIATION PENALTY (Analytic VIF)
  vif_penalty <- 1 / (1 - r2_m_overlap)

  # 3. CORE SENSITIVITY ENGINE
  get_mdcs <- function(cp) {
    r2_grid <- seq(0.0001, 0.8000, by = 0.0001)

    mdcs_balanced <- NA

```

```

mdes_bottleneck <- NA

for(r2 in r2_grid) {
  K <- sqrt(r2)
  min_beta <- K / (max_beta^(k_paths - 1))

  if(min_beta > max_beta) next

  # Compute residual variance of Y
  var_error_y <- 1 - (cp^2 + K^2 + 2 * cp * K) - r2_covariates - r2_parallel
  if(var_error_y <= 0) var_error_y <- 0.01

  # --- BALANCED SCENARIO ---
  beta_bal <- K^(1/k_paths)
  se_bal <- sqrt(var_error_y / (n * (1 - beta_bal^2))) * sqrt(vif_penalty)
  pow_bal <- pnorm((beta_bal / se_bal) - qnorm(1 - alpha/2))
  joint_pow_bal <- pow_bal^k_paths

  # --- BOTTLENECK SCENARIO ---
  se_min <- sqrt(var_error_y / (n * (1 - min_beta^2))) * sqrt(vif_penalty)
  se_max <- sqrt(var_error_y / (n * (1 - max_beta^2))) * sqrt(vif_penalty)

  pow_min <- pnorm((min_beta / se_min) - qnorm(1 - alpha/2))
  pow_max <- pnorm((max_beta / se_max) - qnorm(1 - alpha/2))
  joint_pow_bot <- pow_min * (pow_max^(k_paths - 1))

  if(is.na(mdes_balanced) && joint_pow_bal >= power_target) mdes_balanced
<- r2
  if(is.na(mdes_bottleneck) && joint_pow_bot >= power_target) mdes_bottleneck
<- r2

  if(!is.na(mdes_balanced) && !is.na(mdes_bottleneck)) break
}
return(c(mdes_balanced, mdes_bottleneck))
}

# 4. OUTPUT GENERATION
cat("\n===== \n")
cat("          WRIGHT MEDIATION SENSITIVITY ANALYSIS          \n")
cat("===== \n")
cat(sprintf("Fixed Sample Size (N): %d \n", n))
cat(sprintf("Structural Paths:      %d \n", k_paths))
cat(sprintf("Direct Effect (c'):      %.2f (Base Magnitude)\n", abs(c_prime)))

```

```

cat(sprintf("Other Mediators (R2):  %.2f (Variance in Y)\n", r2_parallel))
cat(sprintf("Target M Overlap (R2):  %.2f (Collinearity VIF = %.2f)\n",
r2_m_overlap, vif_penalty))
cat(sprintf("Covariates (R2):      %.2f \n", r2_covariates))
cat(sprintf("Target Joint Power:   %.2f \n", power_target))
cat("-----\n")
cat(sprintf("%-20s %-18s %-12s\n", "SCENARIO", "MODEL TYPE", "REQ. R2"))
cat("-----\n")

scenarios <- if(type == "explore") c("consistent", "suppression") else type

for(t in scenarios) {
  cp_val <- if(t == "consistent") abs(c_prime) else -abs(c_prime)
  t_label <- if(t == "consistent") "Consistent" else "Suppression"

  results <- get_mdes(cp_val)

  r2_bal <- if(!is.na(results[1])) sprintf("%.4f", results[1]) else
"Unreachable"
  r2_bot <- if(!is.na(results[2])) sprintf("%.4f", results[2]) else
"Unreachable"

  cat(sprintf("%-20s %-18s %-12s\n", "Balanced", t_label, r2_bal))
  cat(sprintf("%-20s %-18s %-12s\n", "Bottleneck", t_label, r2_bot))
  if(t != scenarios[length(scenarios)]) cat("- - - - -\n")
}
cat("=====\n")
cat("Method: Joint Significance Power.\n\n")
}

#####
###Function usage example!

#Single mediator model
wright_sensitivity(n = 107,
  type = "explore",
  k_paths = 2,
  c_prime = 0.30,
  r2_m_overlap = 0,
  r2_parallel = 0,
  r2_covariates = 0,

```

```
power_target = 0.80,
alpha = 0.05,
max_beta = 0.80)
```

```
=====
WRIGHT MEDIATION SENSITIVITY ANALYSIS
=====
```

```
Fixed Sample Size (N): 107
Structural Paths:      2
Direct Effect (c'):    0.30 (Base Magnitude)
Other Mediators (R2):  0.00 (Variance in Y)
Target M Overlap (R2): 0.00 (Collinearity VIF = 1.00)
Covariates (R2):       0.00
Target Joint Power:    0.80
```

SCENARIO	MODEL TYPE	REQ. R2
Balanced	Consistent	0.0081
Bottleneck	Consistent	0.0378
Balanced	Suppression	0.0107
Bottleneck	Suppression	0.0509

```
=====
Method: Joint Significance Power.
```

```
#Two parallel mediators model
wright_sensitivity(n = 105,
                  type = "explore",
                  k_paths = 2,
                  c_prime = 0.30,
                  r2_m_overlap = 0.09,
                  r2_parallel = 0.09,
                  r2_covariates = 0,
                  power_target = 0.80,
                  alpha = 0.05,
                  max_beta = 0.80)
```

```
=====
WRIGHT MEDIATION SENSITIVITY ANALYSIS
=====
```



```
=====
Fixed Sample Size (N): 105
Structural Paths:      2
Direct Effect (c'):    0.30 (Base Magnitude)
Other Mediators (R2):  0.09 (Variance in Y)
Target M Overlap (R2): 0.09 (Collinearity VIF = 1.10)
Covariates (R2):       0.00
Target Joint Power:    0.80
=====
```

```
-----
SCENARIO          MODEL TYPE      REQ. R2
-----
Balanced          Consistent    0.0081
Bottleneck        Consistent    0.0373
-----
Balanced          Suppression    0.0111
Bottleneck        Suppression    0.0520
=====
```

Method: Joint Significance Power.

```
#Two serial mediators model
wright_sensitivity(n = 136,
                  type = "explore",
                  k_paths = 3,
                  r2_m_overlap = 0,
                  r2_parallel = 0,
                  r2_covariates = 0,
                  power_target = 0.80,
                  alpha = 0.05,
                  max_beta = 0.80)
```

WRIGHT MEDIATION SENSITIVITY ANALYSIS

```
=====
Fixed Sample Size (N): 136
Structural Paths:      3
Direct Effect (c'):    0.20 (Base Magnitude)
Other Mediators (R2):  0.00 (Variance in Y)
Target M Overlap (R2): 0.00 (Collinearity VIF = 1.00)
Covariates (R2):       0.00
Target Joint Power:    0.80
=====
```

SCENARIO	MODEL TYPE	REQ. R2
Balanced	Consistent	0.0008
Bottleneck	Consistent	0.0220
Balanced	Suppression	0.0008
Bottleneck	Suppression	0.0252
Method: Joint Significance Power.		